

Rust Data Structures From the Big Picture to Every Line

Assignment 1 | Compilers module | Two implementations, one set of behaviors

What you will understand: how the project fits together, why each data structure behaves as it does, how Rust represents data and ownership, and what every nonblank line of the delivered Rust source means.

The source is in English. This guide is also in English. The walkthrough includes the reusable library, console demonstration, and tests. Blank lines only separate sections; they are not listed as executable instructions.

Verified result: 34 tests passed; formatting passed; Clippy passed with warnings denied; both demonstrations produced the same observable results. Verified with Rust/Cargo 1.70.0.

Reading order: overview → syntax → ownership → operation traces → source walkthrough → tests → exercises and references. You do not need to memorize the syntax before reading the first sections.

The original assignment allows libraries. The additional manual version is included to help you understand what those libraries do for you. The manual dictionary is a search tree, not a hash table.

Contents and reading routes

Click a chapter title to jump to it. The PDF bookmarks provide the same navigation.

Chapter	Page
1. The project from above	3
2. A practical Rust syntax dictionary	4
3. Ownership before algorithms	6
4. Follow the values through the structures	8
5. Library entry point	10
6. The shared contracts	11
7. The standard-collection adapters	16
8. Manual stack, line by line	23
9. Manual queue, line by line	27
10. Manual dictionary, line by line	32
11. The executable demonstration	43
12. The tests, line by line	48
13. Configuration, verification, and your next steps	74
14. Official references and submission checklist	76

If you have 20 minutes

Read chapters 1, 3, and 4; then follow the manual stack in chapter 8 and run cargo run. Focus on take(), Option, and ownership transfer.

If you want to understand every implementation

Read chapters 2 and 6–11. Keep the source file open beside the guide. Trace the linked queue cursor and all three tree-deletion cases by hand.

If you are preparing to explain the assignment

Read chapter 12 to understand what was tested, then complete the exercises in chapter 13. Explain the manual versions' performance limits and the exact scope of AI assistance.

1. The project from above

A data structure organizes values so that selected operations have predictable behavior. Here the behaviors are LIFO, FIFO, and lookup by a unique ordered key. Storage and behavior are separate decisions: two structures can return the same results while using different algorithms.

Concept	Rule	Concrete example
Stack	Last in, first out	Insert 10,20,30; remove 30,20,10.
Queue	First in, first out	Insert 10,20,30; remove 10,20,30.
Dictionary	One value per key	Insert total=int; update total=float; lookup returns float.

Why two versions?

The standard version delegates storage to Vec, VecDeque, and BTreeMap. The manual version builds links and tree operations with nodes. Both implement the same traits, so callers ask for the same operations without depending on the storage layout.

File	Responsibility
Cargo.toml	Project identity, language edition, minimum Rust version, dependencies.
src/lib.rs	Module entry point and unsafe-code prohibition for the library.
src/contracts.rs	Required methods and shared default behavior.
src/std_impl.rs	The three standard-collection adapters.
src/manual_stack.rs	Linked LIFO stack and iterative cleanup.
src/manual_queue.rs	Linked FIFO queue and iterative cleanup.
src/manual_map.rs	Unbalanced search tree, deletion, traversal, and cleanup.
src/main.rs	One generic demonstration instantiated twice.
tests/behavior.rs	Shared scenarios, mixed operations, and destruction counting.

Rust equivalents of classes and interfaces

A struct stores fields; an impl block defines behavior; a trait specifies shared operations. This project does not use class inheritance. Generic functions select implementations at compile time rather than using boxed trait objects.

The file named lib.rs compiles as a reusable library crate. main.rs compiles as a separate binary crate in the same Cargo package and imports that library by its package name. Integration tests are another consumer of the public API.

2. A practical Rust syntax dictionary

Read a signature from left to right: visibility, function name, generic requirements, input parameters, return type, then body. The examples below are drawn from this project.

Syntax	How to read it
pub	Make an item visible outside its module. Fields remain private unless individually declared pub.
struct Node<T>	Define a record type whose value type T is chosen later.
impl<T> Stack<T> for ManualStack<T>	Implement the Stack behavior for this concrete type, for any T.
trait Stack<T>: Default	Implementors must provide the stack methods and also support Default construction.
fn push(&mut self, value: T)	A method that mutably borrows the structure and takes ownership of value.
&self	Shared, read-only access to this instance for the duration of the call.
&mut self	Exclusive mutable access; the method can change links and size.
-> Option<T>	The function returns either Some(owned_value) or None.
Option<&T>	A possible borrowed value: inspecting it does not remove it from storage.
T / K / V	Generic placeholders for item, key, and value types; they are not runtime variables.
K: Ord	Every key must support a total ordering, which lets the tree choose left/equal/right.
usize / i32	usize is a pointer-sized unsigned integer used for counts; i32 is a signed 32-bit integer.
let / let mut	Create an immutable binding / a binding that can be reassigned or mutably borrowed.
Self / self	Self names the current implementation type; self names the current instance.
Self { head: None, size: 0 }	Construct an instance by giving initial values for its fields.
Some(x) / None	The two Option variants: a present value and an absent value.
Box<Node<T>>>	An owning heap pointer with a fixed-size pointer representation; enables recursive nodes.
Option<Box<Node<T>>>>	An optional owned node; None is the end of a list or an empty tree link.
self.head.take()	Move the Option contents out and replace the original field with None.
self.head.take()?	Return None immediately if empty; otherwise unwrap and move out the owned node.
as_ref() / as_mut()	Borrow an Option payload immutably / mutably without taking ownership.
as_deref()	Borrow through the Box, turning Option<Box<Node>> into Option<&Node>.

Syntax	How to read it
<code>.map(node &node.value)</code>	Transform <code>Some(node)</code> into <code>Some(&value)</code> ; leave <code>None</code> unchanged.
<code> key, value ...</code>	An anonymous function (closure), here used to visit dictionary entries.
<code>FnMut(&K, &V)</code>	A callable that receives borrowed keys/values and may mutate captured state.
<code>*link = ...</code>	Dereference a mutable reference and replace the referenced link.
<code>match expression</code>	Choose one branch by comparing against patterns; all possibilities must be covered.
<code>while let Some(node) = link</code>	Repeat while the current <code>Option</code> matches <code>Some</code> ; bind its node.
<code>if let Some(node) = link</code>	Run the block only for the <code>Some</code> variant.
<code>Ordering::Less</code>	A namespaced enum variant returned when the first key is smaller.
<code>std::mem::replace(&mut x, y)</code>	Put <code>y</code> into <code>x</code> and return the old <code>x</code> without requiring a clone.
<code>use crate::contracts::Stack;</code>	Import the <code>Stack</code> trait from the current library crate.
<code>:: / .</code>	<code>::</code> selects a namespace item or associated function; <code>.</code> accesses a field or instance method.
<code>demonstrate::<S,Q,D>(...)</code>	Explicitly choose generic type arguments; <code>::<...></code> is often called turbofish syntax.
<code>&str / String</code>	A borrowed UTF-8 string slice / an owned growable UTF-8 string.
<code>&'static str</code>	A string reference valid for the entire program; literals used in the demo satisfy this.
<code>println!("{:?}", value)</code>	A macro call that prints <code>Debug</code> formatting, useful for <code>Some</code> and <code>None</code> .
<code>assert_eq!(actual, expected)</code>	A test macro that fails by panicking when the two values differ.
<code>#[test]</code>	An attribute marking a function for the Rust test harness.
<code>macro_rules! suite</code>	Define a syntax-expansion macro that generates named test functions.
<code>fn drop(&mut self)</code>	Automatic destruction hook from the <code>Drop</code> trait; cleans up owned nodes.
<code>; / no trailing ;</code>	A semicolon makes an expression a statement; a final expression without one becomes the block result.
<code>// / /// / //!</code>	Ordinary comment / item documentation / enclosing module documentation.
<code>{}, (), []</code>	A block or struct literal; arguments or tuple; array or attribute delimiters, depending on context.

Decode a difficult signature

```
fn visit<F: FnMut(&K, &V)>(&self, mut
visitor: F)
```

`fn visit` defines a method. `F` is the closure type. Its bound says it accepts two shared references and may mutate captured state. `&self` keeps the dictionary read-only. `mut visitor` allows repeated calls that mutate the closure. There is no arrow, so the return type is unit, written `()`.

3. Ownership before algorithms

Each owned value has an owner. Moving a value transfers ownership rather than silently copying it. When an owner leaves scope, Rust drops its value. Borrowing temporarily gives access without transferring ownership. These rules are checked by the compiler. [R1, R2]

The collection stack is not the program call stack

ManualStack is an abstract data structure that obeys LIFO. Its nodes live on the heap because they use Box. The program call stack stores function-call state and is a separate concept. Recursive tree operations consume call-stack space even though tree nodes are heap allocated.

Why Box and Option appear together

A Node cannot directly contain another Node of the same type forever: its size would be infinitely recursive. Box introduces a fixed-size owning pointer. Option lets that pointer be absent. A field of type Option<Box<Node<T>>> means either no next node or one owned next node. [R3, R4]

Why take() is essential

During push, the old head becomes the next node of the new head. During pop, the next node becomes the new head. The method has only a mutable borrow of the structure, so it cannot simply move a non-Copy field out and leave the structure invalid. take() moves the contents out while installing None in its place.

```
let node = self.head.take()?;
self.head = node.next;
self.size -= 1;
Some(node.value)
```

Line 1 obtains an owned boxed node or returns None. Line 2 moves its next link into the stack. Line 3 changes the cached count only after successful removal. Line 4 transfers the stored value to the caller. The detached node allocation is released after its owned fields have been moved; the surviving chain remains owned by self.head.

Borrowing versus removal

peek() returns Option<&T> tied to the shared borrow of the structure. pop() returns Option<T> owned by the caller. If a reference from peek is still being used, a conflicting mutation cannot proceed. Lifetimes are elided in the signatures: the compiler ties the returned borrow to &self.

```
self.head.as_ref().map(|node| &node.value)
```

as_ref() borrows a possible Box; the closure borrows its value field. Rust automatically follows Box/reference dereferences for field access. Nothing is cloned or removed. None bypasses the closure.

A mutable cursor points to a link, not just a node

let mut link = &mut self.head creates an exclusive reference to the head Option. Moving the cursor to &mut node.next makes it refer to the next Option. At the end, *link writes through the reference into the empty link. This lets one insertion algorithm handle both an empty queue and a nonempty one.

Cleanup is part of the implementation

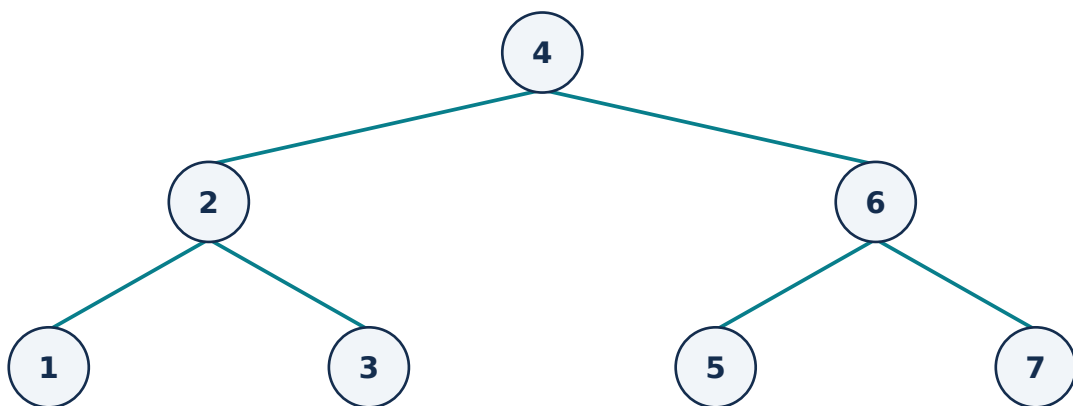
An automatically dropped long chain of boxed nodes could recursively drop the next chain. The manual stack and queue repeatedly remove one node, and the manual tree detaches links iteratively. This avoids recursive node-chain cleanup. Recursive tree visit and deletion still have depth-dependent call-stack limits.

4. Follow the values through the structures

Operation	Head-to-tail / top-to-bottom state	Returned value
Stack push 10	10	()
Stack push 20	20 → 10	()
Stack push 30	30 → 20 → 10	()
Stack peek	30 → 20 → 10	Some(&30)
Stack pop	20 → 10	Some(30)
Queue enqueue 10	10	()
Queue enqueue 20	10 → 20	()
Queue enqueue 30	10 → 20 → 30	()
Queue front	10 → 20 → 30	Some(&10)
Queue dequeue	20 → 30	Some(10)

The dictionary invariant

For every manual tree node, all keys in its left subtree are smaller and all keys in its right subtree are larger. Equal keys update the node value. Searching therefore descends one path; in-order traversal visits left subtree, current node, then right subtree to produce sorted keys.



Insert 4,2,6,1,3,5,7. To find 5: compare with 4 (greater), then 6 (less), then 5 (equal). Traversal returns 1,2,3,4,5,6,7. The shape depends on insertion order because this tree does not rebalance.

Deletion has three structural cases

No children: replace the node link with None. One child: let that child take the old node position. Two children: detach the smallest node of the right subtree, move its key/value into the target node, and reconnect both remaining subtrees. Return the target's original value, not the successor's value.

The tricky fixture inserts 8,4,12,10,11,14. Deleting 8 uses successor 10; successor 10 has right child 11. take_min reconnects 11 where 10 used to be. Losing that reconnection would silently delete a

valid entry; the successor test checks exactly this case.

Iterative tree cleanup

If the root has a left child, promote that child: its right subtree moves into the old root's left link, and the old root becomes its right child. No node is lost. When the current root has no left child, detach its right link, promote it, and drop the now child-free old root. Repeating this process eventually releases every node without recursive destruction.

For root 4 with children 2 and 6, promote 2: 2 becomes root and 4 becomes its right child; the subtree rooted at 3 becomes 4's left subtree. Then process root 2 again. The algorithm temporarily changes shape, but preserves ownership until each node can be safely released.

Behavior versus performance

Operation	Library-backed	Manual
Stack push / pop	$O(1)$ amortized push; $O(1)$ pop	$O(1)$ for both
Queue add / remove	$O(1)$ amortized add; $O(1)$ remove	$O(n)$ add; $O(1)$ remove
Dictionary search / update / delete	$O(\log n)$	$O(h)$; worst case $O(n)$
Ordered visit	$O(n)$	$O(n)$; recursion depth $O(h)$
Clear	$O(n)$	$O(n)$; iterative node cleanup

Amortized means the average cost across a sequence of operations, allowing occasional growth work. h is tree height. Sorted insertion can make h approximately n . The manual versions are learning tools with explicit design limits, not production collection replacements.

5. Library entry point

This short file defines which reusable modules exist. Read it as a map of the library, not as runtime control flow.

Source: `src/lib.rs` | 8 physical lines. Line numbers below are from the final formatted file; blank lines are omitted.

Line	Exact source (wrapped visually)	Meaning and purpose
1	<code>//! Two implementations of the same three data-structure contracts.</code>	Module documentation: states the role or invariant of this file; it is not executed.
2	<code>#![forbid(unsafe_code)]</code>	Crate-level attribute: reject unsafe Rust anywhere in this library, including its modules.
4	<code>pub mod contracts;</code>	Declare and export this library module; Rust loads its matching source file.
5	<code>pub mod manual_map;</code>	Declare and export this library module; Rust loads its matching source file.
6	<code>pub mod manual_queue;</code>	Declare and export this library module; Rust loads its matching source file.
7	<code>pub mod manual_stack;</code>	Declare and export this library module; Rust loads its matching source file.
8	<code>pub mod std_impl;</code>	Declare and export this library module; Rust loads its matching source file.

6. The shared contracts

Each signature defines what both versions must do. Default constructs an empty value. Default methods call required methods so they work in either implementation.

Source: `src/contracts.rs` | 41 physical lines. Line numbers below are from the final formatted file; blank lines are omitted.

Line	Exact source (wrapped visually)	Meaning and purpose
1	<code>//! Shared behavior: callers can switch implementations without changing operations.</code>	Module documentation: states the role or invariant of this file; it is not executed.
3	<code>/// A last-in, first-out collection.</code>	Item documentation: describes the public behavior of the following trait.
4	<code>pub trait Stack<T>: Default {</code>	Define the public <code>Stack<T></code> : behavior contract. Default permits empty construction; method signatures specify the required API.

L5 · push

Line	Exact source (wrapped visually)	Meaning and purpose
5	<code>fn push(&mut self, value: T);</code>	Mutably borrow the stack and take ownership of a new value; return unit. This is a required trait signature with no body here.

L6 · pop

Line	Exact source (wrapped visually)	Meaning and purpose
6	<code>fn pop(&mut self) -> Option<T>;</code>	Mutably borrow the stack and return a removed owned value or None. This is a required trait signature with no body here.

L7 · peek

Line	Exact source (wrapped visually)	Meaning and purpose
7	<code>fn peek(&self) -> Option<&T>;</code>	Borrow the stack read-only and return a possible shared reference to its top. This is a required trait signature with no body here.

L8 · len

Line	Exact source (wrapped visually)	Meaning and purpose
8	<code>fn len(&self) -> usize;</code>	Read the structure and return its entry count as <code>usize</code> . This is a required trait signature with no body here.

L9 · clear

Line	Exact source (wrapped visually)	Meaning and purpose
9	<pre>fn clear(&mut self);</pre>	Mutably borrow the structure and remove all stored values; return unit. This is a required trait signature with no body here.

L10 · is empty

Line	Exact source (wrapped visually)	Meaning and purpose
10	<pre>fn is_empty(&self) -> bool {</pre>	Provide a reusable default method: inspect the collection without changing it and return a boolean.
11	<pre> self.len() == 0</pre>	Return whether the count is zero. No trailing semicolon: this expression is the return value.
12	<pre>}</pre>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
13	<pre>}</pre>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
15	<pre>/// A first-in, first-out collection.</pre>	Item documentation: describes the public behavior of the following trait.
16	<pre>pub trait Queue<T>: Default {</pre>	Define the public Queue<T>: behavior contract. Default permits empty construction; method signatures specify the required API.

L17 · enqueue

Line	Exact source (wrapped visually)	Meaning and purpose
17	<pre>fn enqueue(&mut self, value: T);</pre>	Mutably borrow the queue and take ownership of a value at its back. This is a required trait signature with no body here.

L18 · dequeue

Line	Exact source (wrapped visually)	Meaning and purpose
18	<pre>fn dequeue(&mut self) -> Option<T>;</pre>	Mutably borrow the queue and return its oldest remaining owned value or None. This is a required trait signature with no body here.

L19 · front

Line	Exact source (wrapped visually)	Meaning and purpose
19	<pre>fn front(&self) -> Option<&T>;</pre>	Borrow the queue read-only and return a possible shared reference to its front. This is a required trait signature with no body here.

L20 · len

Line	Exact source (wrapped visually)	Meaning and purpose
20	<pre>fn len(&self) -> usize;</pre>	Read the structure and return its entry count as <code>usize</code> . This is a required trait signature with no body here.

L21 · clear

Line	Exact source (wrapped visually)	Meaning and purpose
21	<pre>fn clear(&mut self);</pre>	Mutably borrow the structure and remove all stored values; return <code>unit</code> . This is a required trait signature with no body here.

L22 · is empty

Line	Exact source (wrapped visually)	Meaning and purpose
22	<pre>fn is_empty(&self) -> bool {</pre>	Provide a reusable default method: inspect the collection without changing it and return a boolean.
23	<pre> self.len() == 0</pre>	Return whether the count is zero. No trailing semicolon: this expression is the return value.
24	<pre>}</pre>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
25	<pre>}</pre>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
27	<pre>/// Unique keys, replacement on duplicate insertion, and sorted traversal.</pre>	Item documentation: describes the public behavior of the following trait.
28	<pre>pub trait Dictionary<K: Ord, V>: Default {</pre>	Define the public <code>Dictionary<K></code> behavior contract. Default permits empty construction; method signatures specify the required API.

L29 · insert

Line	Exact source (wrapped visually)	Meaning and purpose
29	<pre>fn insert(&mut self, key: K, value: V) -> Option<V>;</pre>	Take ownership of a key/value pair; return the old owned value only when replacing an existing key. This is a required trait signature with no body here.

L30 · get

Line	Exact source (wrapped visually)	Meaning and purpose
30	<pre>fn get(&self, key: &K) -> Option<&V>;</pre>	Borrow the search key and structure; return a possible shared reference to the stored value. This is a required trait signature with no body here.

L31 · remove

Line	Exact source (wrapped visually)	Meaning and purpose
31	<pre>fn remove(&mut self, key: &K) -> Option<V>;</pre>	Borrow the search key, mutate the structure, and return a removed owned value or None. This is a required trait signature with no body here.

L32 · visit

Line	Exact source (wrapped visually)	Meaning and purpose
32	<pre>fn visit<F: FnMut(&K, &V)>(&self, visitor: F);</pre>	Borrow the dictionary and accept a FnMut callback to receive key/value references in order. This is a required trait signature with no body here.

L33 · len

Line	Exact source (wrapped visually)	Meaning and purpose
33	<pre>fn len(&self) -> usize;</pre>	Read the structure and return its entry count as usize. This is a required trait signature with no body here.

L34 · clear

Line	Exact source (wrapped visually)	Meaning and purpose
34	<pre>fn clear(&mut self);</pre>	Mutably borrow the structure and remove all stored values; return unit. This is a required trait signature with no body here.

L35 · contains key

Line	Exact source (wrapped visually)	Meaning and purpose
35	<pre>fn contains_key(&self, key: &K) -> bool {</pre>	Default membership method: borrow a key and return whether it has an associated value.
36	<pre> self.get(key).is_some()</pre>	Reuse get; return true for Some and false for None, without removing the value.
37	<pre>}</pre>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L38 · is empty

Line	Exact source (wrapped visually)	Meaning and purpose
38	<code>fn is_empty(&self) -> bool {</code>	Provide a reusable default method: inspect the collection without changing it and return a boolean.
39	<code> self.len() == 0</code>	Return whether the count is zero. No trailing semicolon: this expression is the return value.
40	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
41	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

7. The standard-collection adapters

These methods are deliberately small: they translate our common operation names into standard collection operations. Private items fields prevent callers from bypassing the intended stack/queue rules.

Source: `src/std_impl.rs` | 90 physical lines. Line numbers below are from the final formatted file; blank lines are omitted.

Line	Exact source (wrapped visually)	Meaning and purpose
1	<code>//! Adapters around Rust's standard collections; no external crates are needed.</code>	Module documentation: states the role or invariant of this file; it is not executed.
2	<code>use crate::contracts::{Dictionary, Queue, Stack};</code>	Import all three contracts before implementing them for the adapters.
3	<code>use std::collections::{BTreeMap, VecDeque};</code>	Import the standard ordered map and double-ended queue. Vec is already in Rust's prelude.
5	<code>pub struct LibraryStack<T> {</code>	Define a record type. Generic parameters choose item/key/value types; pub exposes the type while its fields remain private.
6	<code> items: Vec<T>,</code>	Store stack items in a standard growable contiguous vector.
7	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
8	<code>impl<T> Default for LibraryStack<T> {</code>	Implement empty construction explicitly, avoiding an unnecessary Default requirement on stored values.

L9 · default

Line	Exact source (wrapped visually)	Meaning and purpose
9	<code>fn default() -> Self {</code>	Construct a fresh empty instance, returned as Self; no receiver is needed.
10	<code> Self { items: Vec::new() }</code>	Construct an empty adapter by constructing its inner vector.
11	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
12	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
13	<code>impl<T> Stack<T> for LibraryStack<T> {</code>	Implement the named shared trait for this concrete structure. Generic bounds supply any required key ordering.

L14 · push

Line	Exact source (wrapped visually)	Meaning and purpose
14	<code>fn push(&mut self, value: T) {</code>	Mutably borrow the stack and take ownership of a new value; return unit.
15	<code>self.items.push(value);</code>	Delegate addition to the inner Vec, which owns the new value.
16	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L17 · pop

Line	Exact source (wrapped visually)	Meaning and purpose
17	<code>fn pop(&mut self) -> Option<T> {</code>	Mutably borrow the stack and return a removed owned value or None.
18	<code>self.items.pop()</code>	Delegate removal to Vec; forward its Option result as the method result.
19	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L20 · peek

Line	Exact source (wrapped visually)	Meaning and purpose
20	<code>fn peek(&self) -> Option<&T> {</code>	Borrow the stack read-only and return a possible shared reference to its top.
21	<code>self.items.last()</code>	Borrow the Vec's final item without removing it; forward Option<&T>.
22	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L23 · len

Line	Exact source (wrapped visually)	Meaning and purpose
23	<code>fn len(&self) -> usize {</code>	Read the structure and return its entry count as usize.
24	<code>self.items.len()</code>	Return the inner collection's count.
25	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L26 · clear

Line	Exact source (wrapped visually)	Meaning and purpose
26	<code>fn clear(&mut self) {</code>	Mutably borrow the structure and remove all stored values; return unit.
27	<code> self.items.clear();</code>	Remove all entries from the inner standard collection.
28	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
29	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
31	<code>pub struct LibraryQueue<T> {</code>	Define a record type. Generic parameters choose item/key/value types; pub exposes the type while its fields remain private.
32	<code> items: VecDeque<T>,</code>	Store queue items in the standard growable ring-buffer deque.
33	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
34	<code>impl<T> Default for LibraryQueue<T> {</code>	Implement empty construction explicitly, avoiding an unnecessary Default requirement on stored values.

L35 · default

Line	Exact source (wrapped visually)	Meaning and purpose
35	<code>fn default() -> Self {</code>	Construct a fresh empty instance, returned as Self; no receiver is needed.
36	<code> Self {</code>	Start a struct literal for the concrete type named by this impl block.
37	<code> items: VecDeque::new(),</code>	Create the empty deque used as this queue's storage.
38	<code> }</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
39	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
40	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
41	<code>impl<T> Queue<T> for LibraryQueue<T> {</code>	Implement the named shared trait for this concrete structure. Generic bounds supply any required key ordering.

L42 · enqueue

Line	Exact source (wrapped visually)	Meaning and purpose
42	<code>fn enqueue(&mut self, value: T) {</code>	Mutably borrow the queue and take ownership of a value at its back.
43	<code>self.items.push_back(value);</code>	Delegate insertion at the deque's back to preserve FIFO order.
44	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L45 · dequeue

Line	Exact source (wrapped visually)	Meaning and purpose
45	<code>fn dequeue(&mut self) -> Option<T> {</code>	Mutably borrow the queue and return its oldest remaining owned value or None.
46	<code>self.items.pop_front()</code>	Delegate removal from the deque's front; forward its owned Option result.
47	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L48 · front

Line	Exact source (wrapped visually)	Meaning and purpose
48	<code>fn front(&self) -> Option<&T> {</code>	Borrow the queue read-only and return a possible shared reference to its front.
49	<code>self.items.front()</code>	Borrow the deque's first item without removing it.
50	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L51 · len

Line	Exact source (wrapped visually)	Meaning and purpose
51	<code>fn len(&self) -> usize {</code>	Read the structure and return its entry count as usize.
52	<code>self.items.len()</code>	Return the inner collection's count.
53	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L54 · clear

Line	Exact source (wrapped visually)	Meaning and purpose
54	<code>fn clear(&mut self) {</code>	Mutably borrow the structure and remove all stored values; return unit.
55	<code>self.items.clear();</code>	Remove all entries from the inner standard collection.
56	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
57	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
59	<code>pub struct LibraryDictionary<K, V> {</code>	Define a record type. Generic parameters choose item/key/value types; pub exposes the type while its fields remain private.
60	<code>items: BTreeMap<K, V>,</code>	Store key/value entries in the standard ordered B-tree map.
61	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
62	<code>impl<K, V> Default for LibraryDictionary<K, V> {</code>	Implement empty construction explicitly, avoiding an unnecessary Default requirement on stored values.

L63 · default

Line	Exact source (wrapped visually)	Meaning and purpose
63	<code>fn default() -> Self {</code>	Construct a fresh empty instance, returned as Self; no receiver is needed.
64	<code>Self {</code>	Start a struct literal for the concrete type named by this impl block.
65	<code>items: BTreeMap::new(),</code>	Create the empty B-tree map used as this dictionary's storage.
66	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
67	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
68	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
69	<code>impl<K: Ord, V> Dictionary<K, V> for LibraryDictionary<K, V> {</code>	Implement the named shared trait for this concrete structure. Generic bounds supply any required key ordering.

L70 · insert

Line	Exact source (wrapped visually)	Meaning and purpose
70	<pre>fn insert(&mut self, key: K, value: V) -> Option<V> {</pre>	Take ownership of a key/value pair; return the old owned value only when replacing an existing key.
71	<pre> self.items.insert(key, value)</pre>	Delegate key insertion/replacement to BTreeMap; forward its previous-value result.
72	<pre>}</pre>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L73 · get

Line	Exact source (wrapped visually)	Meaning and purpose
73	<pre>fn get(&self, key: &K) -> Option<&V> {</pre>	Borrow the search key and structure; return a possible shared reference to the stored value.
74	<pre> self.items.get(key)</pre>	Delegate borrowed lookup to BTreeMap; forward the borrowed Option result.
75	<pre>}</pre>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L76 · remove

Line	Exact source (wrapped visually)	Meaning and purpose
76	<pre>fn remove(&mut self, key: &K) -> Option<V> {</pre>	Borrow the search key, mutate the structure, and return a removed owned value or None.
77	<pre> self.items.remove(key)</pre>	Delegate key deletion to BTreeMap; forward its removed owned value.
78	<pre>}</pre>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L79 · visit

Line	Exact source (wrapped visually)	Meaning and purpose
79	<pre>fn visit<F: FnMut(&K, &V)>(&self, mut visitor: F) {</pre>	Borrow the dictionary and accept a FnMut callback to receive key/value references in order.
80	<pre> for (key, value) in &self.items {</pre>	Borrow and iterate through the BTreeMap entries; its iterator yields sorted keys.
81	<pre> visitor(key, value);</pre>	Invoke the caller's mutable callback with the borrowed pair.
82	<pre> }</pre>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
83	<pre>}</pre>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L84 · len

Line	Exact source (wrapped visually)	Meaning and purpose
84	<pre>fn len(&self) -> usize {</pre>	Read the structure and return its entry count as usize.
85	<pre> self.items.len()</pre>	Return the inner collection's count.
86	<pre>}</pre>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L87 · clear

Line	Exact source (wrapped visually)	Meaning and purpose
87	<pre>fn clear(&mut self) {</pre>	Mutably borrow the structure and remove all stored values; return unit.
88	<pre> self.items.clear();</pre>	Remove all entries from the inner standard collection.
89	<pre>}</pre>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
90	<pre>}</pre>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

8. Manual stack, line by line

Invariant: head owns the newest node, each next link owns the node below it, and size equals the number of reachable nodes. push changes one link; pop detaches one node.

Source: src/manual_stack.rs | 46 physical lines. Line numbers below are from the final formatted file; blank lines are omitted.

Line	Exact source (wrapped visually)	Meaning and purpose
1	<code>//! A manually linked stack. Each node owns the node below it.</code>	Module documentation: states the role or invariant of this file; it is not executed.
2	<code>use crate::contracts::Stack;</code>	Bring the stack trait into scope, including its methods used during cleanup.
4	<code>struct Node<T> {</code>	Define a record type. Generic parameters choose item/key/value types; pub exposes the type while its fields remain private.
5	<code> value: T,</code>	A stored item of generic type T. The node owns it; no Copy or Clone bound is required.
6	<code> next: Option<Box<Node<T>>>>,</code>	Optional owned link to the following node. None terminates the list.
7	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
8	<code>pub struct ManualStack<T> {</code>	Define a record type. Generic parameters choose item/key/value types; pub exposes the type while its fields remain private.
9	<code> head: Option<Box<Node<T>>>,</code>	Optional first node: stack top or queue front. It owns the entire linked chain.
10	<code> size: usize,</code>	Cache the number of stored items so length checks do not need a traversal.
11	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
12	<code>impl<T> Default for ManualStack<T> {</code>	Implement empty construction explicitly, avoiding an unnecessary Default requirement on stored values.

L13 · default

Line	Exact source (wrapped visually)	Meaning and purpose
13	<code>fn default() -> Self {</code>	Construct a fresh empty instance, returned as Self; no receiver is needed.
14	<code>Self {</code>	Start a struct literal for the concrete type named by this impl block.
15	<code>head: None,</code>	Initialize the list with no first node.
16	<code>size: 0,</code>	Initialize the cached item count to zero.
17	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
18	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
19	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
20	<code>impl<T> Stack<T> for ManualStack<T> {</code>	Implement the named shared trait for this concrete structure. Generic bounds supply any required key ordering.

L21 · push

Line	Exact source (wrapped visually)	Meaning and purpose
21	<code>fn push(&mut self, value: T) {</code>	Mutably borrow the stack and take ownership of a new value; return unit.
22	<code>let next = self.head.take();</code>	Move the old head out while replacing it with None; save that old chain as next.
23	<code>self.head = Some(Box::new(Node { value, next }));</code>	Allocate a new node that owns value and the previous head; make it the new stack top.
24	<code>self.size += 1;</code>	One new item was linked successfully; increment the cached count exactly once.
25	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L26 · pop

Line	Exact source (wrapped visually)	Meaning and purpose
26	<code>fn pop(&mut self) -> Option<T> {</code>	Mutably borrow the stack and return a removed owned value or None.
27	<code> let node = self.head.take()?;</code>	Take the first owned node. If the head is None, ? returns None immediately and leaves size unchanged.
28	<code> self.head = node.next;</code>	Move the detached node's next link into head so the remaining chain stays owned.
29	<code> self.size -= 1;</code>	Successful removal guarantees at least one item existed; decrement the cached count.
30	<code> Some(node.value)</code>	Move the removed node's value to the caller as the block result.
31	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L32 · peek

Line	Exact source (wrapped visually)	Meaning and purpose
32	<code>fn peek(&self) -> Option<&T> {</code>	Borrow the stack read-only and return a possible shared reference to its top.
33	<code> self.head.as_ref().map(node &node.value)</code>	Borrow the head and transform a present node into a reference to its value; None remains None.
34	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L35 · len

Line	Exact source (wrapped visually)	Meaning and purpose
35	<code>fn len(&self) -> usize {</code>	Read the structure and return its entry count as usize.
36	<code> self.size</code>	Return the cached count without traversal or mutation.
37	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L38 · clear

Line	Exact source (wrapped visually)	Meaning and purpose
38	<code>fn clear(&mut self) {</code>	Mutably borrow the structure and remove all stored values; return unit.
39	<code> while self.pop().is_some() {}</code>	Repeatedly remove and drop one stack value until empty; the empty body has no extra work.
40	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
41	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
42	<code>impl<T> Drop for ManualStack<T> {</code>	Implement automatic cleanup when this owner leaves scope or is explicitly dropped.

L43 · drop

Line	Exact source (wrapped visually)	Meaning and purpose
43	<code>fn drop(&mut self) {</code>	Automatic Drop hook, borrowing the soon-to-be-destroyed instance mutably.
44	<code> self.clear();</code>	Reuse iterative collection cleanup during automatic destruction.
45	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
46	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

9. Manual queue, line by line

Invariant: head owns the oldest remaining node and next links preserve arrival order. enqueue walks to an empty tail link. This design deliberately has $O(n)$ insertion and uses no unsafe tail pointer.

Source: `src/manual_queue.rs` | 50 physical lines. Line numbers below are from the final formatted file; blank lines are omitted.

Line	Exact source (wrapped visually)	Meaning and purpose
1	<code>//! A simple linked FIFO queue: enqueue is $O(n)$, dequeue is $O(1)$.</code>	Module documentation: states the role or invariant of this file; it is not executed.
2	<code>use crate::contracts::Queue;</code>	Bring the queue trait into scope, including its methods used during cleanup.
4	<code>struct Node<T> {</code>	Define a record type. Generic parameters choose item/key/value types; pub exposes the type while its fields remain private.
5	<code> value: T,</code>	A stored item of generic type T. The node owns it; no Copy or Clone bound is required.
6	<code> next: Option<Box<Node<T>>>>,</code>	Optional owned link to the following node. None terminates the list.
7	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
8	<code>pub struct ManualQueue<T> {</code>	Define a record type. Generic parameters choose item/key/value types; pub exposes the type while its fields remain private.
9	<code> head: Option<Box<Node<T>>>,</code>	Optional first node: stack top or queue front. It owns the entire linked chain.
10	<code> size: usize,</code>	Cache the number of stored items so length checks do not need a traversal.
11	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
12	<code>impl<T> Default for ManualQueue<T> {</code>	Implement empty construction explicitly, avoiding an unnecessary Default requirement on stored values.

L13 · default

Line	Exact source (wrapped visually)	Meaning and purpose
13	<code>fn default() -> Self {</code>	Construct a fresh empty instance, returned as Self; no receiver is needed.
14	<code>Self {</code>	Start a struct literal for the concrete type named by this impl block.
15	<code>head: None,</code>	Initialize the list with no first node.
16	<code>size: 0,</code>	Initialize the cached item count to zero.
17	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
18	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
19	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
20	<code>impl<T> Queue<T> for ManualQueue<T> {</code>	Implement the named shared trait for this concrete structure. Generic bounds supply any required key ordering.

L21 · enqueue

Line	Exact source (wrapped visually)	Meaning and purpose
21	<code>fn enqueue(&mut self, value: T) {</code>	Mutably borrow the queue and take ownership of a value at its back.
22	<code> // Walk through mutable links until reaching the empty tail link.</code>	Implementation comment: explains the intention of the following code; comments do not run.
23	<code> let mut link = &mut self.head;</code>	Create an exclusive cursor to the head link. The cursor will move to successive next links.
24	<code> while let Some(node) = link {</code>	Continue while the mutable link has a node; bind its mutable node reference, then advance toward the empty tail link.
25	<code> link = &mut node.next;</code>	Advance the exclusive cursor to the next Option field; do not move or clone the node.
26	<code> }</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
27	<code> *link = Some(Box::new(Node { value, next: None }));</code>	Write a new terminal node into the empty link, handling both the empty-list and tail-insertion cases.
28	<code> self.size += 1;</code>	One new item was linked successfully; increment the cached count exactly once.
29	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L30 · dequeue

Line	Exact source (wrapped visually)	Meaning and purpose
30	<code>fn dequeue(&mut self) -> Option<T> {</code>	Mutably borrow the queue and return its oldest remaining owned value or None.
31	<code> let node = self.head.take()?;</code>	Take the first owned node. If the head is None, ? returns None immediately and leaves size unchanged.
32	<code> self.head = node.next;</code>	Move the detached node's next link into head so the remaining chain stays owned.
33	<code> self.size -= 1;</code>	Successful removal guarantees at least one item existed; decrement the cached count.
34	<code> Some(node.value)</code>	Move the removed node's value to the caller as the block result.
35	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L36 · front

Line	Exact source (wrapped visually)	Meaning and purpose
36	<code>fn front(&self) -> Option<T> {</code>	Borrow the queue read-only and return a possible shared reference to its front.
37	<code> self.head.as_ref().map(node &node.value)</code>	Borrow the head and transform a present node into a reference to its value; None remains None.
38	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L39 · len

Line	Exact source (wrapped visually)	Meaning and purpose
39	<code>fn len(&self) -> usize {</code>	Read the structure and return its entry count as usize.
40	<code> self.size</code>	Return the cached count without traversal or mutation.
41	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L42 · clear

Line	Exact source (wrapped visually)	Meaning and purpose
42	<code>fn clear(&mut self) {</code>	Mutably borrow the structure and remove all stored values; return unit.
43	<code> while self.dequeue().is_some() {}</code>	Repeatedly remove and drop one queue value until empty; each removal detaches one node.
44	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
45	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
46	<code>impl<T> Drop for ManualQueue<T> {</code>	Implement automatic cleanup when this owner leaves scope or is explicitly dropped.

L47 · drop

Line	Exact source (wrapped visually)	Meaning and purpose
47	<code>fn drop(&mut self) {</code>	Automatic Drop hook, borrowing the soon-to-be-destroyed instance mutably.
48	<code>self.clear();</code>	Reuse iterative collection cleanup during automatic destruction.
49	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
50	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

10. Manual dictionary, line by line

Invariant: every left subtree has smaller keys, every right subtree has larger keys, and size counts unique keys. Focus first on insert/get, then remove_node/take_min, then ordered traversal and iterative cleanup.

Source: src/manual_map.rs | 146 physical lines. Line numbers below are from the final formatted file; blank lines are omitted.

Line	Exact source (wrapped visually)	Meaning and purpose
1	<code>//! An unbalanced binary search tree: left keys < key < right keys.</code>	Module documentation: states the role or invariant of this file; it is not executed.
2	<code>use crate::contracts::Dictionary;</code>	Bring the dictionary trait into scope so this module can implement its required methods.
3	<code>use std::cmp::Ordering;</code>	Import the comparison enum with Less, Equal, and Greater variants.
5	<code>type Link<K, V> = Option<Box<Node<K, V>>>;</code>	Create a type alias, not a new type: an absent link or a uniquely owned boxed tree node.
6	<code>struct Node<K, V> {</code>	Define a record type. Generic parameters choose item/key/value types; pub exposes the type while its fields remain private.
7	<code> key: K,</code>	This node owns its key of generic type K.
8	<code> value: V,</code>	This node owns the value associated with its key.
9	<code> left: Link<K, V>,</code>	Owned link to the subtree containing keys smaller than this node's key.
10	<code> right: Link<K, V>,</code>	Owned link to the subtree containing keys larger than this node's key.
11	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
12	<code>pub struct ManualDictionary<K, V> {</code>	Define a record type. Generic parameters choose item/key/value types; pub exposes the type while its fields remain private.
13	<code> root: Link<K, V>,</code>	Owned root link. None represents an empty dictionary.
14	<code> size: usize,</code>	Cache the number of stored items so length checks do not need a traversal.
15	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
16	<code>impl<K, V> Default for ManualDictionary<K, V> {</code>	Implement empty construction explicitly, avoiding an unnecessary Default requirement on stored values.

L17 · default

Line	Exact source (wrapped visually)	Meaning and purpose
17	<code>fn default() -> Self {</code>	Construct a fresh empty instance, returned as Self; no receiver is needed.
18	<code>Self {</code>	Start a struct literal for the concrete type named by this impl block.
19	<code>root: None,</code>	Initialize the tree with no root node.
20	<code>size: 0,</code>	Initialize the cached item count to zero.
21	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
22	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
23	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
25	<code>// Detach the smallest node; its right child takes its former position.</code>	Implementation comment: explains the intention of the following code; comments do not run.

L26 · take min

Precondition: this helper receives a nonempty subtree. It detaches one node and preserves that node's former right subtree.

Line	Exact source (wrapped visually)	Meaning and purpose
26	<code>fn take_min<K, V>(link: &mut Link<K, V>) -> Box<Node<K, V>> {</code>	Private helper: mutate a nonempty subtree link and return its detached smallest boxed node.
27	<code>if link.as_ref().expect("nonempty subtree").left.is_some() {</code>	Borrow the current node and check whether a smaller node exists on its left. The helper requires a nonempty subtree.
28	<code>return take_min(&mut link.as_mut().unwrap().left);</code>	Recurse through the left link. unwrap is valid because the previous condition established this link is Some.
29	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
30	<code>let mut node = link.take().expect("nonempty subtree");</code>	Detach the smallest node. expect checks the internal nonempty-subtree invariant, not an external key lookup.
31	<code>*link = node.right.take();</code>	Promote the smallest node's right child into its old position, preserving that subtree.
32	<code>node</code>	Return the detached boxed node as the final expression.

Line	Exact source (wrapped visually)	Meaning and purpose
33	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L35 · remove node

Only this helper changes tree shape on deletion. The public remove method adjusts size once based on its result.

Line	Exact source (wrapped visually)	Meaning and purpose
35	fn remove_node<K: Ord, V>(link: &mut Link<K, V>, key: &K) -> Option<V> {	Recursive helper: mutate an owned subtree link and search by borrowed ordered key; return a possible removed value.
36	match key.cmp(&link.as_ref()?.key) {	Compare the search key with the borrowed node key. An absent link causes ? to return None.
37	Ordering::Less => remove_node(&mut link.as_mut()?.left, key),	Smaller key: recurse into the left link and return that call's result.
38	Ordering::Greater => remove_node(&mut link.as_mut()?.right, key),	Larger key: recurse into the right link and return that call's result.
39	Ordering::Equal => {	The keys match; this branch updates or removes the current node, depending on the enclosing function.
40	let mut node = link.take()?;	Detach the matched node from its parent/root link so its owned fields can be rearranged.
41	match (node.left.take(), node.right.take()) {	Detach both child links and match their tuple to select the appropriate deletion case.
42	(None, right) => {	No left child: this also covers a leaf if right is None.
43	*link = right;	The right child (possibly absent) takes the removed node's position.
44	Some(node.value)	Move the removed node's value to the caller as the block result.
45	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
46	(left, None) => {	There is a left child and no right child; the earlier arm already handled both absent.
47	*link = left;	The left subtree takes the removed node's position.
48	Some(node.value)	Move the removed node's value to the caller as the block result.
49	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

Line	Exact source (wrapped visually)	Meaning and purpose
50	<code>(left, right) => {</code>	Both children are present; previous match arms excluded the absent-child cases.
51	<code>let mut subtree = right;</code>	Make the detached right subtree mutable so its smallest node can be removed.
52	<code>let successor = take_min(&mut subtree);</code>	Detach the in-order successor, preserving its possible right child inside subtree.
53	<code>node.key = successor.key;</code>	Move the successor key into the target node. The original matched key is dropped.
54	<code>let old = std::mem::replace(&mut node.value, successor.value);</code>	Move in the successor value and retain the original target value for the caller.
55	<code>node.left = left;</code>	Reconnect the unchanged left subtree to the replacement-key node.
56	<code>node.right = subtree;</code>	Reconnect the right subtree after successor removal.
57	<code>*link = Some(node);</code>	Reattach the rebuilt node to its original parent/root link.
58	<code>Some(old)</code>	Return the original target value, not the successor's value.
59	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
60	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
61	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
62	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
63	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L65 · visit node

Line	Exact source (wrapped visually)	Meaning and purpose
65	<pre>fn visit_node<K, V, F: FnMut(&K, &V)>(link: &Link<K, V>, visitor: &mut F) {</pre>	Recursive helper: borrow a subtree and the same mutable callback for in-order traversal.
66	<pre> if let Some(node) = link {</pre>	If the borrowed subtree is nonempty, bind a shared reference to its node; otherwise do nothing.
67	<pre> visit_node(&node.left, visitor);</pre>	Visit all smaller keys first; pass along the same mutable callback.
68	<pre> visitor(&node.key, &node.value);</pre>	Call the visitor with borrowed data from the current node.
69	<pre> visit_node(&node.right, visitor);</pre>	Visit all larger keys last, completing in-order traversal.
70	<pre> }</pre>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
71	<pre>}</pre>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
73	<pre>impl<K: Ord, V> Dictionary<K, V> for ManualDictionary<K, V> {</pre>	Implement the named shared trait for this concrete structure. Generic bounds supply any required key ordering.

L74 · insert

Line	Exact source (wrapped visually)	Meaning and purpose
74	<code>fn insert(&mut self, key: K, value: V) -> Option<V> {</code>	Take ownership of a key/value pair; return the old owned value only when replacing an existing key.
75	<code> let mut link = &mut self.root;</code>	Start an exclusive link cursor at the tree root; insertion will descend without recursion.
76	<code> loop {</code>	Repeat until insertion or replacement explicitly returns.
77	<code> match link {</code>	Inspect the current mutable Option link: either an empty position or an existing node.
78	<code> None => {</code>	An empty link was reached; this is the insertion position for the new key.
79	<code> *link = Some(Box::new(Node {</code>	Dereference the cursor and begin allocating the new node in the empty link.
80	<code> key,</code>	Field shorthand: move the local variable key into the field also named key.
81	<code> value,</code>	Field shorthand: move the local variable value into the field also named value.
82	<code> left: None,</code>	A newly inserted tree node begins with an empty left subtree.
83	<code> right: None,</code>	A newly inserted tree node begins with an empty right subtree.
84	<code> }));</code>	Finish the node literal, heap allocation, Some wrapper, and link-assignment statement.
85	<code> self.size += 1;</code>	One new item was linked successfully; increment the cached count exactly once.
86	<code> return None;</code>	A new key was added, so there is no previous value to return.
87	<code> }</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
88	<code> Some(node) => match key.cmp(&node.key) {</code>	At an existing node, compare keys to descend left/right or replace an equal key's value.
89	<code> Ordering::Less => link = &mut node.left,</code>	Smaller key: move the mutable cursor to the left link.
90	<code> Ordering::Greater => link = &mut node.right,</code>	Larger key: move the mutable cursor to the right link.
91	<code> Ordering::Equal => {</code>	The keys match; this branch updates or removes the current node, depending on the enclosing function.
92	<code> return Some(std::mem::replace(&mut node.value, value));</code>	Replace the value for an existing key and return the old value. Do not increment size.

Line	Exact source (wrapped visually)	Meaning and purpose
93	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
94	},	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
95	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
96	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
97	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L98 · get

Line	Exact source (wrapped visually)	Meaning and purpose
98	<code>fn get(&self, key: &K) -> Option<&V> {</code>	Borrow the search key and structure; return a possible shared reference to the stored value.
99	<code> let mut link = self.root.as_deref();</code>	Borrow the root as <code>Option<&Node></code> rather than borrowing the <code>Box</code> wrapper.
100	<code> while let Some(node) = link {</code>	Continue while a borrowed node exists; each loop compares the desired key with this node.
101	<code> match key.cmp(&node.key) {</code>	Compare the requested key with the current node key to choose a search direction.
102	<code> Ordering::Less => link = node.left.as_deref(),</code>	Smaller key: borrow the left node for the next iteration.
103	<code> Ordering::Greater => link = node.right.as_deref(),</code>	Larger key: borrow the right node for the next iteration.
104	<code> Ordering::Equal => return Some(&node.value),</code>	Found the key: return a reference tied to the dictionary borrow, without moving the value.
105	<code> }</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
106	<code> }</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
107	<code> None</code>	The search reached an absent link, so return a missing result.
108	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L109 · remove

Line	Exact source (wrapped visually)	Meaning and purpose
109	<pre>fn remove(&mut self, key: &K) -> Option<V> {</pre>	Borrow the search key, mutate the structure, and return a removed owned value or None.
110	<pre> let removed = remove_node(&mut self.root, key);</pre>	Delegate structural deletion to the helper; retain its possible owned value.
111	<pre> if removed.is_some() {</pre>	Only decrement size if a key was actually removed.
112	<pre> self.size -= 1;</pre>	Successful removal guarantees at least one item existed; decrement the cached count.
113	<pre> }</pre>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
114	<pre> removed</pre>	Return the deletion result unchanged, preserving ownership of its value.
115	<pre>}</pre>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L116 · visit

Line	Exact source (wrapped visually)	Meaning and purpose
116	<pre>fn visit<F: FnMut(&K, &V)>(&self, mut visitor: F) {</pre>	Borrow the dictionary and accept a FnMut callback to receive key/value references in order.
117	<pre> visit_node(&self.root, &mut visitor);</pre>	Start the recursive ordered traversal with one mutable visitor instance.
118	<pre>}</pre>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L119 · len

Line	Exact source (wrapped visually)	Meaning and purpose
119	<pre>fn len(&self) -> usize {</pre>	Read the structure and return its entry count as usize.
120	<pre> self.size</pre>	Return the cached count without traversal or mutation.
121	<pre>}</pre>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L122 · clear

Line	Exact source (wrapped visually)	Meaning and purpose
122	<code>fn clear(&mut self) {</code>	Mutably borrow the structure and remove all stored values; return unit.
123	<code>self.clear_nodes();</code>	Run the bound-free iterative cleanup helper, which also resets size.
124	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
125	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
127	<code>impl<K, V> ManualDictionary<K, V> {</code>	Start an inherent implementation block for private methods on this type, independent of a shared trait.
128	<code>// Rotate left children upward, then drop child-free nodes iteratively.</code>	Implementation comment: explains the intention of the following code; comments do not run.

L129 · clear nodes

The type has no Ord bound here because cleanup does not compare keys. Temporary rotations preserve ownership and permit nonrecursive destruction.

Line	Exact source (wrapped visually)	Meaning and purpose
129	<code>fn clear_nodes(&mut self) {</code>	Private, bound-free cleanup helper: no key comparisons are needed to release the tree.
130	<code>while let Some(mut node) = self.root.take() {</code>	Detach the current owned root; continue until no node remains.
131	<code>if let Some(mut left) = node.left.take() {</code>	Detach a left child if present; it will be promoted to preserve the remaining tree.
132	<code>node.left = left.right.take();</code>	Move the promoted child's former right subtree into the old root's left link.
133	<code>left.right = Some(node);</code>	Attach the old root as the promoted child's right child; all remaining nodes are still owned.
134	<code>self.root = Some(left);</code>	Make the promoted child the temporary root and continue cleanup.
135	<code>} else {</code>	Choose the alternative branch when there is no left child.
136	<code>self.root = node.right.take();</code>	Detach/promote the right link. The old root now has no children and drops without recursively dropping descendants.
137	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

Line	Exact source (wrapped visually)	Meaning and purpose
138	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
139	self.size = 0;	All nodes have been removed; reset the cached count.
140	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
141	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
142	impl<K, V> Drop for ManualDictionary<K, V> {	Implement automatic cleanup when this owner leaves scope or is explicitly dropped.

L143 · drop

Line	Exact source (wrapped visually)	Meaning and purpose
143	fn drop(&mut self) {	Automatic Drop hook, borrowing the soon-to-be-destroyed instance mutably.
144	self.clear_nodes();	Run the bound-free iterative cleanup helper, which also resets size.
145	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
146	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

11. The executable demonstration

demonstrate is generic over three structure types. Its operations do not change when main selects the other implementation family. String literals provide the static references used as symbols.

Source: src/main.rs | 70 physical lines. Line numbers below are from the final formatted file; blank lines are omitted.

Line	Exact source (wrapped visually)	Meaning and purpose
1	<code>//! Run identical examples with both implementations.</code>	Module documentation: states the role or invariant of this file; it is not executed.
2	<code>use rust_structures_assignment::contracts::{Dictionary, Queue, Stack};</code>	Import public library items into this binary/test crate. Traits must also be in scope to call their methods.
3	<code>use rust_structures_assignment::manual_map::ManualDictionary;</code>	Import public library items into this binary/test crate. Traits must also be in scope to call their methods.
4	<code>use rust_structures_assignment::manual_queue::ManualQueue;</code>	Import public library items into this binary/test crate. Traits must also be in scope to call their methods.
5	<code>use rust_structures_assignment::manual_stack::ManualStack;</code>	Import public library items into this binary/test crate. Traits must also be in scope to call their methods.
6	<code>use rust_structures_assignment::std_impl::{LibraryDictionary, LibraryQueue, LibraryStack};</code>	Import public library items into this binary/test crate. Traits must also be in scope to call their methods.

L8 · demonstrate

Line	Exact source (wrapped visually)	Meaning and purpose
8	<code>fn demonstrate<S, Q, D>(label: &str)</code>	Generic console example: S/Q/D select three implementation types; label is a borrowed string.
9	<code>where</code>	Begin explicit generic trait bounds for the demonstration function.
10	<code>S: Stack<i32>,</code>	Require the selected stack implementation to store 32-bit integers.
11	<code>Q: Queue<i32>,</code>	Require the selected queue implementation to store 32-bit integers.
12	<code>D: Dictionary<&'static str, &'static str>,</code>	Require a dictionary of long-lived string references; the demonstration uses string literals.
13	<code>{</code>	Open a block; local owners created here are dropped when its scope ends.
14	<code>println!("\n=== {label} ===");</code>	Print this labeled observation. <code>{:?}</code> formats Option/debug values; <code>{}</code> uses display formatting; <code>println!</code> also adds a newline.
15	<code>let mut stack = S::default();</code>	Construct an empty instance of the selected stack type; <code>mut</code> allows its operations to change it.
16	<code>let mut queue = Q::default();</code>	Construct an empty instance of the selected queue type.
17	<code>for value in [10, 20, 30] {</code>	Iterate over a fixed array in this order; each integer is Copy and can be used in both structures.
18	<code>stack.push(value);</code>	Add this integer to the selected stack implementation.
19	<code>queue.enqueue(value);</code>	Add the same integer to the selected queue implementation.
20	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
21	<code>println!("Stack top: {:?}; size: {}", stack.peek(), stack.len());</code>	Print this labeled observation. <code>{:?}</code> formats Option/debug values; <code>{}</code> uses display formatting; <code>println!</code> also adds a newline.
22	<code>println!("Stack removal order:");</code>	Print this labeled observation. <code>{:?}</code> formats Option/debug values; <code>{}</code> uses display formatting; <code>println!</code> also adds a newline.
23	<code>while let Some(value) = stack.pop() {</code>	Repeatedly remove stack items; stop when pop returns None.
24	<code>println!(" {value}");</code>	Print the removed integer with a preceding space, using captured-variable formatting and no newline.
25	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

Line	Exact source (wrapped visually)	Meaning and purpose
26	<code>println!();</code>	Print a newline to finish the sequence already written with <code>print!</code> .
27	<code>println!("Queue front: {:?}; size: {}", queue.front(), queue.len());</code>	Print this labeled observation. <code>{:?}</code> formats Option/debug values; <code>{}</code> uses display formatting; <code>println!</code> also adds a newline.
28	<code>print!("Queue removal order:");</code>	Print this labeled observation. <code>{:?}</code> formats Option/debug values; <code>{}</code> uses display formatting; <code>println!</code> also adds a newline.
29	<code>while let Some(value) = queue.dequeue() {</code>	Repeatedly remove queue items; stop when dequeue returns <code>None</code> .
30	<code>print!(" {}value");</code>	Print the removed integer with a preceding space, using captured-variable formatting and no newline.
31	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
32	<code>println!();</code>	Print a newline to finish the sequence already written with <code>print!</code> .
33	<code>println!(</code>	Start a multi-line print macro; the format string and arguments follow.
34	<code> "Empty removal: stack={:?}, queue={:?}"</code>	Format-string argument for the surrounding print macro. Its placeholders are filled by the following expressions.
35	<code> stack.pop(),</code>	Supply the empty stack removal result to the print macro: <code>None</code> .
36	<code> queue.dequeue()</code>	Supply the empty queue removal result to the print macro: <code>None</code> .
37	<code>);</code>	Close the multi-line macro/function call and terminate the statement.
38	<code>let mut symbols = D::default();</code>	Construct the chosen dictionary; it models a tiny variable-name/type table.
39	<code>symbols.insert("total", "int");</code>	Add the symbol <code>total</code> with type <code>int</code> .
40	<code>symbols.insert("active", "bool");</code>	Add <code>active</code> after <code>total</code> ; traversal will still place <code>active</code> first by key order.
41	<code>symbols.insert("name", "string");</code>	Add a third key to demonstrate later removal.
42	<code>println!("Lookup total: {:?}", symbols.get(&"total"));</code>	Print this labeled observation. <code>{:?}</code> formats Option/debug values; <code>{}</code> uses display formatting; <code>println!</code> also adds a newline.
43	<code>println!(</code>	Start a multi-line print macro; the format string and arguments follow.
44	<code> "Update total; old type: {:?}",</code>	Format-string argument for the surrounding print macro. Its placeholders are filled by the following expressions.

Line	Exact source (wrapped visually)	Meaning and purpose
45	<code>symbols.insert("total", "float")</code>	Update an existing key and pass its old value to the surrounding print macro.
46	<code>);</code>	Close the multi-line macro/function call and terminate the statement.
47	<code>println!("Contains name: {}", symbols.contains_key(&"name"));</code>	Print this labeled observation. <code>{:?}</code> formats Option/debug values; <code>{}</code> uses display formatting; <code>println!</code> also adds a newline.
48	<code>println!("Remove name: {:?}", symbols.remove(&"name"));</code>	Print this labeled observation. <code>{:?}</code> formats Option/debug values; <code>{}</code> uses display formatting; <code>println!</code> also adds a newline.
49	<code>println!("Missing lookup: {:?}", symbols.get(&"missing"));</code>	Print this labeled observation. <code>{:?}</code> formats Option/debug values; <code>{}</code> uses display formatting; <code>println!</code> also adds a newline.
50	<code>println!("Dictionary in key order ({ entries):", symbols.len());</code>	Print this labeled observation. <code>{:?}</code> formats Option/debug values; <code>{}</code> uses display formatting; <code>println!</code> also adds a newline.
51	<code>symbols.visit(key, value println!("{}", {key} -> {value}"));</code>	Traverse in sorted order; a closure prints each borrowed key/value pair.
52	<code>stack.push(99);</code>	Add an item after draining to demonstrate reuse before clearing.
53	<code>queue.enqueue(99);</code>	Add an item after draining to demonstrate reuse before clearing.
54	<code>stack.clear();</code>	Remove any remaining stack values.
55	<code>queue.clear();</code>	Remove any remaining queue values.
56	<code>symbols.clear();</code>	Remove all dictionary entries.
57	<code>println!(</code>	Start a multi-line print macro; the format string and arguments follow.
58	<code> "After clear: stack={}, queue={}, dictionary={}",</code>	Format-string argument for the surrounding print macro. Its placeholders are filled by the following expressions.
59	<code> stack.is_empty(),</code>	Supply the stack empty check to the formatted output.
60	<code> queue.is_empty(),</code>	Supply the queue empty check to the formatted output.
61	<code> symbols.is_empty()</code>	Supply the dictionary empty check to the formatted output.
62	<code>);</code>	Close the multi-line macro/function call and terminate the statement.
63	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L65 · main

Line	Exact source (wrapped visually)	Meaning and purpose
65	<code>fn main() {</code>	Binary entry point: run the generic demonstration once for each implementation family.
66	<code> demonstrate::<LibraryStack<i32>, LibraryQueue<i32>, LibraryDictionary<&str, &str>>("STANDARD LIBRARY",);</code>	Choose concrete stack, queue, and dictionary types using explicit generic arguments; execute the shared demonstration.
67		Provide the heading string for the library-backed demonstration.
68		Close the multi-line macro/function call and terminate the statement.
69	<code> demonstrate::<ManualStack<i32>, ManualQueue<i32>, ManualDictionary<&str, &str>>("MANUAL NODES");</code>	Choose concrete stack, queue, and dictionary types using explicit generic arguments; execute the shared demonstration.
70	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

12. The tests, line by line

Sixteen scenario helpers are expanded twice by the suite macro. The remaining two tests compare 1,000 mixed steps and count destruction of values. Vec/Rc/Cell here are observation tools, not manual collection storage.

Source: tests/behavior.rs | 323 physical lines. Line numbers below are from the final formatted file; blank lines are omitted.

Line	Exact source (wrapped visually)	Meaning and purpose
1	<code>//! The same behavioral checks run against both implementations.</code>	Module documentation: states the role or invariant of this file; it is not executed.
2	<code>use rust_structures_assignment::contracts::{Dictionary, Queue, Stack};</code>	Import public library items into this binary/test crate. Traits must also be in scope to call their methods.
3	<code>use rust_structures_assignment::manual_map::ManualDictionary;</code>	Import public library items into this binary/test crate. Traits must also be in scope to call their methods.
4	<code>use rust_structures_assignment::manual_queue::ManualQueue;</code>	Import public library items into this binary/test crate. Traits must also be in scope to call their methods.
5	<code>use rust_structures_assignment::manual_stack::ManualStack;</code>	Import public library items into this binary/test crate. Traits must also be in scope to call their methods.
6	<code>use rust_structures_assignment::std_impl::{LibraryDictionary, LibraryQueue, LibraryStack};</code>	Import public library items into this binary/test crate. Traits must also be in scope to call their methods.

L8 · stack empty

Scenario purpose: validate empty-stack behavior without count underflow.

Line	Exact source (wrapped visually)	Meaning and purpose
8	<code>fn stack_empty<S: Stack<i32>>>() {</code>	Generic test helper to validate empty-stack behavior without count underflow. Its type bound permits either implementation.
9	<code>let mut s = S::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
10	<code>assert!(s.is_empty());</code>	Require this boolean condition to be true; ! negates membership when present. Context: validate empty-stack behavior without count underflow.
11	<code>assert_eq!(s.len(), 0);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate empty-stack behavior without count underflow.
12	<code>assert_eq!(s.peek(), None);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate empty-stack behavior without count underflow.

Line	Exact source (wrapped visually)	Meaning and purpose
13	<code>assert_eq!(s.pop(), None);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate empty-stack behavior without count underflow.
14	<code>assert_eq!(s.len(), 0);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate empty-stack behavior without count underflow.
15	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L16 · stack order

Scenario purpose: validate LIFO order and a non-removing peek.

Line	Exact source (wrapped visually)	Meaning and purpose
16	<code>fn stack_order<S: Stack<i32>>() {</code>	Generic test helper to validate LIFO order and a non-removing peek. Its type bound permits either implementation.
17	<code>let mut s = S::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
18	<code>for value in [10, 20, 30] {</code>	Iterate over the insertion sequence 10, 20, 30 to establish the expected order for this single test structure.
19	<code>s.push(value);</code>	Insert the specified owned value into the selected implementation; duplicates are intentional in reuse scenarios.
20	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
21	<code>assert_eq!(s.peek(), Some(&30));</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate LIFO order and a non-removing peek.
22	<code>assert_eq!(s.len(), 3);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate LIFO order and a non-removing peek.
23	<code>for value in [30, 20, 10] {</code>	Check removed stack items against the expected reverse insertion order.
24	<code>assert_eq!(s.pop(), Some(value));</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate LIFO order and a non-removing peek.
25	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

Line	Exact source (wrapped visually)	Meaning and purpose
26	<code>assert!(s.is_empty());</code>	Require this boolean condition to be true; ! negates membership when present. Context: validate LIFO order and a non-removing peek.
27	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L28 · stack clear

Scenario purpose: validate duplicates, repeated clearing, and reuse.

Line	Exact source (wrapped visually)	Meaning and purpose
28	<code>fn stack_clear<S: Stack<i32>>() {</code>	Generic test helper to validate duplicates, repeated clearing, and reuse. Its type bound permits either implementation.
29	<code>let mut s = S::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
30	<code>s.push(1);</code>	Insert the specified owned value into the selected implementation; duplicates are intentional in reuse scenarios.
31	<code>s.push(1);</code>	Insert the specified owned value into the selected implementation; duplicates are intentional in reuse scenarios.
32	<code>assert_eq!(s.len(), 2);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate duplicates, repeated clearing, and reuse.
33	<code>s.clear();</code>	Clear the test instance. Calling this twice checks that clearing an empty structure remains safe.
34	<code>s.clear();</code>	Clear the test instance. Calling this twice checks that clearing an empty structure remains safe.
35	<code>assert_eq!(s.pop(), None);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate duplicates, repeated clearing, and reuse.
36	<code>s.push(9);</code>	Insert the specified owned value into the selected implementation; duplicates are intentional in reuse scenarios.
37	<code>assert_eq!(s.pop(), Some(9));</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate duplicates, repeated clearing, and reuse.
38	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L39 · stack owned

Scenario purpose: validate an owned String without requiring Clone.

Line	Exact source (wrapped visually)	Meaning and purpose
39	<code>fn stack_owned<S: Stack<String>>() {</code>	Generic test helper to validate an owned String without requiring Clone. Its type bound permits either implementation.
40	<code>let mut s = S::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
41	<code>s.push(String::from("owned"));</code>	Insert the specified owned value into the selected implementation; duplicates are intentional in reuse scenarios.
42	<code>assert_eq!(s.peek().map(String::as_str), Some("owned"));</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate an owned String without requiring Clone.
43	<code>assert_eq!(s.pop(), Some(String::from("owned")));</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate an owned String without requiring Clone.
44	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L45 · queue empty

Scenario purpose: validate empty-queue behavior.

Line	Exact source (wrapped visually)	Meaning and purpose
45	<code>fn queue_empty<Q: Queue<i32>>() {</code>	Generic test helper to validate empty-queue behavior. Its type bound permits either implementation.
46	<code>let mut q = Q::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
47	<code>assert!(q.is_empty());</code>	Require this boolean condition to be true; ! negates membership when present. Context: validate empty-queue behavior.
48	<code>assert_eq!(q.len(), 0);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate empty-queue behavior.
49	<code>assert_eq!(q.front(), None);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate empty-queue behavior.
50	<code>assert_eq!(q.dequeue(), None);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate empty-queue behavior.

Line	Exact source (wrapped visually)	Meaning and purpose
51	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L52 · queue order

Scenario purpose: validate FIFO order and a non-removing front query.

Line	Exact source (wrapped visually)	Meaning and purpose
52	fn queue_order<Q: Queue<i32>>() {	Generic test helper to validate FIFO order and a non-removing front query. Its type bound permits either implementation.
53	let mut q = Q::default();	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
54	for value in [10, 20, 30] {	Iterate over the insertion sequence 10, 20, 30 to establish the expected order for this single test structure.
55	q.enqueue(value);	Insert the specified owned value into the selected implementation; duplicates are intentional in reuse scenarios.
56	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
57	assert_eq!(q.front(), Some(&10));	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate FIFO order and a non-removing front query.
58	assert_eq!(q.len(), 3);	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate FIFO order and a non-removing front query.
59	for value in [10, 20, 30] {	Iterate over the insertion sequence 10, 20, 30 to establish the expected order for this single test structure.
60	assert_eq!(q.dequeue(), Some(value));	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate FIFO order and a non-removing front query.
61	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
62	assert!(q.is_empty());	Require this boolean condition to be true; ! negates membership when present. Context: validate FIFO order and a non-removing front query.
63	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L64 · queue interleaved

Scenario purpose: validate FIFO ordering when enqueue and dequeue alternate.

Line	Exact source (wrapped visually)	Meaning and purpose
64	<code>fn queue_interleaved<Q: Queue<i32>>() {</code>	Generic test helper to validate FIFO ordering when enqueue and dequeue alternate. Its type bound permits either implementation.
65	<code>let mut q = Q::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
66	<code>q.enqueue(1);</code>	Insert the specified owned value into the selected implementation; duplicates are intentional in reuse scenarios.
67	<code>q.enqueue(2);</code>	Insert the specified owned value into the selected implementation; duplicates are intentional in reuse scenarios.
68	<code>assert_eq!(q.dequeue(), Some(1));</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate FIFO ordering when enqueue and dequeue alternate.
69	<code>q.enqueue(3);</code>	Insert the specified owned value into the selected implementation; duplicates are intentional in reuse scenarios.
70	<code>assert_eq!(q.dequeue(), Some(2));</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate FIFO ordering when enqueue and dequeue alternate.
71	<code>assert_eq!(q.dequeue(), Some(3));</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate FIFO ordering when enqueue and dequeue alternate.
72	<code>q.enqueue(4);</code>	Insert the specified owned value into the selected implementation; duplicates are intentional in reuse scenarios.
73	<code>assert_eq!(q.dequeue(), Some(4));</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate FIFO ordering when enqueue and dequeue alternate.
74	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L75 · queue clear

Scenario purpose: validate duplicates, repeated clearing, and reuse.

Line	Exact source (wrapped visually)	Meaning and purpose
75	<code>fn queue_clear<Q: Queue<i32>>() {</code>	Generic test helper to validate duplicates, repeated clearing, and reuse. Its type bound permits either implementation.

Line	Exact source (wrapped visually)	Meaning and purpose
76	<code>let mut q = Q::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
77	<code>q.enqueue(1);</code>	Insert the specified owned value into the selected implementation; duplicates are intentional in reuse scenarios.
78	<code>q.enqueue(1);</code>	Insert the specified owned value into the selected implementation; duplicates are intentional in reuse scenarios.
79	<code>assert_eq!(q.len(), 2);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate duplicates, repeated clearing, and reuse.
80	<code>q.clear();</code>	Clear the test instance. Calling this twice checks that clearing an empty structure remains safe.
81	<code>q.clear();</code>	Clear the test instance. Calling this twice checks that clearing an empty structure remains safe.
82	<code>assert!(q.is_empty());</code>	Require this boolean condition to be true; ! negates membership when present. Context: validate duplicates, repeated clearing, and reuse.
83	<code>q.enqueue(9);</code>	Insert the specified owned value into the selected implementation; duplicates are intentional in reuse scenarios.
84	<code>assert_eq!(q.dequeue(), Some(9));</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate duplicates, repeated clearing, and reuse.
85	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L86 · queue owned

Scenario purpose: validate an owned String without requiring Clone.

Line	Exact source (wrapped visually)	Meaning and purpose
86	<code>fn queue_owned<Q: Queue<String>>() {</code>	Generic test helper to validate an owned String without requiring Clone. Its type bound permits either implementation.
87	<code>let mut q = Q::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
88	<code>q.enqueue(String::from("owned"));</code>	Insert the specified owned value into the selected implementation; duplicates are intentional in reuse scenarios.

Line	Exact source (wrapped visually)	Meaning and purpose
89	<code>assert_eq!(q.front().map(String::as_str), Some("owned"));</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate an owned String without requiring Clone.
90	<code>assert_eq!(q.dequeue(), Some(String::from("owned")));</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate an owned String without requiring Clone.
91	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L92 · dictionary empty

Scenario purpose: validate missing keys and empty traversal.

Line	Exact source (wrapped visually)	Meaning and purpose
92	<code>fn dictionary_empty<D: Dictionary<i32, i32>>() {</code>	Generic test helper to validate missing keys and empty traversal. Its type bound permits either implementation.
93	<code>let mut d = D::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
94	<code>assert!(d.is_empty());</code>	Require this boolean condition to be true; ! negates membership when present. Context: validate missing keys and empty traversal.
95	<code>assert_eq!(d.get(&1), None);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate missing keys and empty traversal.
96	<code>assert!(!d.contains_key(&1));</code>	Require this boolean condition to be true; ! negates membership when present. Context: validate missing keys and empty traversal.
97	<code>assert_eq!(d.remove(&1), None);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate missing keys and empty traversal.
98	<code>let mut calls = 0;</code>	Initialize the count of visitor invocations; an empty dictionary must keep it at zero.
99	<code>d.visit(_, _ calls += 1);</code>	Invoke ordered traversal with a closure that records keys/pairs or increments the test-only visitor counter.
100	<code>assert_eq!(calls, 0);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate missing keys and empty traversal.
101	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L102 · dictionary update

Scenario purpose: validate replacement, count preservation, and repeated removal.

Line	Exact source (wrapped visually)	Meaning and purpose
102	<pre>fn dictionary_update<D: Dictionary<i32, i32>>() {</pre>	Generic test helper to validate replacement, count preservation, and repeated removal. Its type bound permits either implementation.
103	<pre> let mut d = D::default();</pre>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
104	<pre> assert_eq!(d.insert(2, 20), None);</pre>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate replacement, count preservation, and repeated removal.
105	<pre> assert_eq!(d.insert(2, 99), Some(20));</pre>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate replacement, count preservation, and repeated removal.
106	<pre> assert_eq!(d.len(), 1);</pre>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate replacement, count preservation, and repeated removal.
107	<pre> assert_eq!(d.get(&2), Some(&99));</pre>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate replacement, count preservation, and repeated removal.
108	<pre> assert!(d.contains_key(&2));</pre>	Require this boolean condition to be true; ! negates membership when present. Context: validate replacement, count preservation, and repeated removal.
109	<pre> assert_eq!(d.remove(&2), Some(99));</pre>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate replacement, count preservation, and repeated removal.
110	<pre> assert_eq!(d.remove(&2), None);</pre>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate replacement, count preservation, and repeated removal.
111	<pre> assert!(d.is_empty());</pre>	Require this boolean condition to be true; ! negates membership when present. Context: validate replacement, count preservation, and repeated removal.
112	<pre>}</pre>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L113 · dictionary order

Scenario purpose: validate complete ascending-key traversal.

Line	Exact source (wrapped visually)	Meaning and purpose
113	<pre>fn dictionary_order<D: Dictionary<i32, i32>>() {</pre>	Generic test helper to validate complete ascending-key traversal. Its type bound permits either implementation.

Line	Exact source (wrapped visually)	Meaning and purpose
114	<code>let mut d = D::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
115	<code>for key in [4, 2, 6, 1, 3, 5, 7] {</code>	Insert keys in this explicit order to create the intended tree shape; later assertions inspect order or deletion behavior.
116	<code> d.insert(key, key * 10);</code>	Insert the specified test key/value; type inference follows the generic scenario or tracked-value fixture.
117	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
118	<code>let mut entries = Vec::new();</code>	Create a test-only vector that collects observed traversal output; the manual storage itself does not use it.
119	<code> d.visit(key, value entries.push((*key, *value)));</code>	Invoke ordered traversal with a closure that records keys/pairs or increments the test-only visitor counter.
120	<code> assert_eq!(entries, (1..=7).map(k (k, k * 10)).collect::<Vec<_>>());</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate complete ascending-key traversal.
121	<code> assert_eq!(d.len(), 7);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate complete ascending-key traversal.
122	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L123 · dictionary removal

Scenario purpose: exercise all child-count deletion cases and remaining counts.

Line	Exact source (wrapped visually)	Meaning and purpose
123	<code>fn dictionary_removal<D: Dictionary<i32, i32>>() {</code>	Generic test helper to exercise all child-count deletion cases and remaining counts. Its type bound permits either implementation.
124	<code> let mut d = D::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
125	<code> for key in [4, 2, 6, 1, 3, 5, 7] {</code>	Insert keys in this explicit order to create the intended tree shape; later assertions inspect order or deletion behavior.
126	<code> d.insert(key, key * 10);</code>	Insert the specified test key/value; type inference follows the generic scenario or tracked-value fixture.
127	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

Line	Exact source (wrapped visually)	Meaning and purpose
128	// Leaf, one-child, and two-child deletions, including the root.	Implementation comment: explains the intention of the following code; comments do not run.
129	for (index, key) in [1, 2, 4, 6, 3, 5, 7].into_iter().enumerate() {	Iterate through the deletion sequence with a zero-based index to compute the expected remaining count.
130	assert_eq!(d.remove(&key), Some(key * 10));	Require actual and expected values to be equal; failure panics and marks this test failed. Context: exercise all child-count deletion cases and remaining counts.
131	assert_eq!(d.get(&key), None);	Require actual and expected values to be equal; failure panics and marks this test failed. Context: exercise all child-count deletion cases and remaining counts.
132	assert_eq!(d.len(), 6 - index);	Require actual and expected values to be equal; failure panics and marks this test failed. Context: exercise all child-count deletion cases and remaining counts.
133	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
134	assert!(d.is_empty());	Require this boolean condition to be true; ! negates membership when present. Context: exercise all child-count deletion cases and remaining counts.
135	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L136 · dictionary successor

Scenario purpose: preserve the successor's right child during two-child deletion.

Line	Exact source (wrapped visually)	Meaning and purpose
136	fn dictionary_successor<D: Dictionary<i32, i32>>() {	Generic test helper to preserve the successor's right child during two-child deletion. Its type bound permits either implementation.
137	let mut d = D::default();	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
138	for key in [8, 4, 12, 10, 11, 14] {	Insert keys in this explicit order to create the intended tree shape; later assertions inspect order or deletion behavior.
139	d.insert(key, key);	Insert the specified test key/value; type inference follows the generic scenario or tracked-value fixture.
140	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
141	assert_eq!(d.remove(&8), Some(8));	Require actual and expected values to be equal; failure panics and marks this test failed. Context: preserve the successor's right child during two-child deletion.

Line	Exact source (wrapped visually)	Meaning and purpose
142	<code>let mut keys = Vec::new();</code>	Create a test-only vector that collects observed traversal output; the manual storage itself does not use it.
143	<code>d.visit(key, _ keys.push(*key));</code>	Invoke ordered traversal with a closure that records keys/pairs or increments the test-only visitor counter.
144	<code>assert_eq!(keys, [4, 10, 11, 12, 14]);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: preserve the successor's right child during two-child deletion.
145	<code>assert_eq!(d.len(), 5);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: preserve the successor's right child during two-child deletion.
146	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L147 · dictionary clear

Scenario purpose: validate cleanup and reuse after ascending-key insertion.

Line	Exact source (wrapped visually)	Meaning and purpose
147	<code>fn dictionary_clear<D: Dictionary<i32, i32>>() {</code>	Generic test helper to validate cleanup and reuse after ascending-key insertion. Its type bound permits either implementation.
148	<code>let mut d = D::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
149	<code>for key in 0..128 {</code>	Insert 128 ascending keys (0 through 127), creating a skewed manual tree for the cleanup/reuse test.
150	<code>d.insert(key, key);</code>	Insert the specified test key/value; type inference follows the generic scenario or tracked-value fixture.
151	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
152	<code>d.clear();</code>	Clear all dictionary entries; the test later confirms count, absence of old keys, and reuse.
153	<code>d.clear();</code>	Clear all dictionary entries; the test later confirms count, absence of old keys, and reuse.
154	<code>assert_eq!(d.len(), 0);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate cleanup and reuse after ascending-key insertion.
155	<code>assert_eq!(d.get(&127), None);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate cleanup and reuse after ascending-key insertion.

Line	Exact source (wrapped visually)	Meaning and purpose
156	<code>d.insert(9, 90);</code>	Insert the specified test key/value; type inference follows the generic scenario or tracked-value fixture.
157	<code>assert_eq!(d.remove(&9), Some(90));</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate cleanup and reuse after ascending-key insertion.
158	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

L159 · dictionary owned

Scenario purpose: validate owned string keys and values without cloning.

Line	Exact source (wrapped visually)	Meaning and purpose
159	<code>fn dictionary_owned<D: Dictionary<String, String>>() {</code>	Generic test helper to validate owned string keys and values without cloning. Its type bound permits either implementation.
160	<code>let mut d = D::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
161	<code>d.insert(String::from("name"), String::from("string"));</code>	Insert the specified test key/value; type inference follows the generic scenario or tracked-value fixture.
162	<code>assert_eq!(</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate owned string keys and values without cloning.
163	<code>d.get(&String::from("name")).map(String::as_str),</code>	Borrow the owned string-key entry and map its String value to &str for content comparison in the assertion.
164	<code>Some("string")</code>	Expected borrowed string slice inside Some for the owned-dictionary lookup assertion.
165	<code>);</code>	Close the multi-line macro/function call and terminate the statement.
166	<code>assert_eq!(</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: validate owned string keys and values without cloning.
167	<code>d.remove(&String::from("name")),</code>	Remove by a temporary borrowed owned-string key and pass the result to the surrounding assertion.
168	<code>Some(String::from("string"))</code>	Construct the expected owned String result inside Some for the surrounding equality assertion.
169	<code>);</code>	Close the multi-line macro/function call and terminate the statement.
170	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

Line	Exact source (wrapped visually)	Meaning and purpose
172	// Generate named test wrappers while sharing each scenario's actual assertions.	Implementation comment: explains the intention of the following code; comments do not run.
173	macro_rules! suite {	Define the suite syntax macro; its expansion will generate 16 separately named test functions per implementation.
174	(\$s:ident, \$q:ident, \$d:ident) => {	Macro pattern: accept three type identifiers and expand the body with those names substituted.
175	#[test]	Mark the following zero-argument function as a test for cargo test to discover.

L176 · empty stack

Line	Exact source (wrapped visually)	Meaning and purpose
176	fn empty_stack() {	Named test wrapper generated by the suite macro; its body selects one concrete type and invokes a shared scenario.
177	stack_empty::<\$s<i32>>();	Invoke this shared scenario with the concrete type substituted by the suite macro; <...> chooses stored value types.
178	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
179	#[test]	Mark the following zero-argument function as a test for cargo test to discover.

L180 · lifo stack

Line	Exact source (wrapped visually)	Meaning and purpose
180	fn lifo_stack() {	Named test wrapper generated by the suite macro; its body selects one concrete type and invokes a shared scenario.
181	stack_order::<\$s<i32>>();	Invoke this shared scenario with the concrete type substituted by the suite macro; <...> chooses stored value types.
182	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
183	#[test]	Mark the following zero-argument function as a test for cargo test to discover.

L184 · reusable stack

Line	Exact source (wrapped visually)	Meaning and purpose
184	<code>fn reusable_stack() {</code>	Named test wrapper generated by the suite macro; its body selects one concrete type and invokes a shared scenario.
185	<code>stack_clear::<\$s<i32>>>();</code>	Invoke this shared scenario with the concrete type substituted by the suite macro; <...> chooses stored value types.
186	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
187	<code>#[test]</code>	Mark the following zero-argument function as a test for cargo test to discover.

L188 · owned stack

Line	Exact source (wrapped visually)	Meaning and purpose
188	<code>fn owned_stack() {</code>	Named test wrapper generated by the suite macro; its body selects one concrete type and invokes a shared scenario.
189	<code>stack_owned::<\$s<String>>>();</code>	Invoke this shared scenario with the concrete type substituted by the suite macro; <...> chooses stored value types.
190	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
191	<code>#[test]</code>	Mark the following zero-argument function as a test for cargo test to discover.

L192 · empty queue

Line	Exact source (wrapped visually)	Meaning and purpose
192	<code>fn empty_queue() {</code>	Named test wrapper generated by the suite macro; its body selects one concrete type and invokes a shared scenario.
193	<code>queue_empty::<\$q<i32>>>();</code>	Invoke this shared scenario with the concrete type substituted by the suite macro; <...> chooses stored value types.
194	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
195	<code>#[test]</code>	Mark the following zero-argument function as a test for cargo test to discover.

L196 · fifo queue

Line	Exact source (wrapped visually)	Meaning and purpose
196	<code>fn fifo_queue() {</code>	Named test wrapper generated by the suite macro; its body selects one concrete type and invokes a shared scenario.
197	<code>queue_order::<\$q<i32>>();</code>	Invoke this shared scenario with the concrete type substituted by the suite macro; <...> chooses stored value types.
198	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
199	<code>#[test]</code>	Mark the following zero-argument function as a test for cargo test to discover.

L200 · mixed queue

Line	Exact source (wrapped visually)	Meaning and purpose
200	<code>fn mixed_queue() {</code>	Named test wrapper generated by the suite macro; its body selects one concrete type and invokes a shared scenario.
201	<code>queue_interleaved::<\$q<i32>>();</code>	Invoke this shared scenario with the concrete type substituted by the suite macro; <...> chooses stored value types.
202	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
203	<code>#[test]</code>	Mark the following zero-argument function as a test for cargo test to discover.

L204 · reusable queue

Line	Exact source (wrapped visually)	Meaning and purpose
204	<code>fn reusable_queue() {</code>	Named test wrapper generated by the suite macro; its body selects one concrete type and invokes a shared scenario.
205	<code>queue_clear::<\$q<i32>>();</code>	Invoke this shared scenario with the concrete type substituted by the suite macro; <...> chooses stored value types.
206	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
207	<code>#[test]</code>	Mark the following zero-argument function as a test for cargo test to discover.

L208 · owned queue

Line	Exact source (wrapped visually)	Meaning and purpose
208	<code>fn owned_queue() {</code>	Named test wrapper generated by the suite macro; its body selects one concrete type and invokes a shared scenario.
209	<code>queue_owned::<\$q<String>>();</code>	Invoke this shared scenario with the concrete type substituted by the suite macro; <...> chooses stored value types.
210	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
211	<code>#[test]</code>	Mark the following zero-argument function as a test for cargo test to discover.

L212 · empty dictionary

Line	Exact source (wrapped visually)	Meaning and purpose
212	<code>fn empty_dictionary() {</code>	Named test wrapper generated by the suite macro; its body selects one concrete type and invokes a shared scenario.
213	<code>dictionary_empty::<\$d<i32, i32>>();</code>	Invoke this shared scenario with the concrete type substituted by the suite macro; <...> chooses stored value types.
214	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
215	<code>#[test]</code>	Mark the following zero-argument function as a test for cargo test to discover.

L216 · updated dictionary

Line	Exact source (wrapped visually)	Meaning and purpose
216	<code>fn updated_dictionary() {</code>	Named test wrapper generated by the suite macro; its body selects one concrete type and invokes a shared scenario.
217	<code>dictionary_update::<\$d<i32, i32>>();</code>	Invoke this shared scenario with the concrete type substituted by the suite macro; <...> chooses stored value types.
218	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
219	<code>#[test]</code>	Mark the following zero-argument function as a test for cargo test to discover.

L220 · sorted dictionary

Line	Exact source (wrapped visually)	Meaning and purpose
220	<code>fn sorted_dictionary() {</code>	Named test wrapper generated by the suite macro; its body selects one concrete type and invokes a shared scenario.
221	<code> dictionary_order::<\$d<i32, i32>>());</code>	Invoke this shared scenario with the concrete type substituted by the suite macro; <...> chooses stored value types.
222	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
223	<code>#[test]</code>	Mark the following zero-argument function as a test for cargo test to discover.

L224 · deleted dictionary

Line	Exact source (wrapped visually)	Meaning and purpose
224	<code>fn deleted_dictionary() {</code>	Named test wrapper generated by the suite macro; its body selects one concrete type and invokes a shared scenario.
225	<code> dictionary_removal::<\$d<i32, i32>>());</code>	Invoke this shared scenario with the concrete type substituted by the suite macro; <...> chooses stored value types.
226	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
227	<code>#[test]</code>	Mark the following zero-argument function as a test for cargo test to discover.

L228 · successor dictionary

Line	Exact source (wrapped visually)	Meaning and purpose
228	<code>fn successor_dictionary() {</code>	Named test wrapper generated by the suite macro; its body selects one concrete type and invokes a shared scenario.
229	<code> dictionary_successor::<\$d<i32, i32>>());</code>	Invoke this shared scenario with the concrete type substituted by the suite macro; <...> chooses stored value types.
230	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
231	<code>#[test]</code>	Mark the following zero-argument function as a test for cargo test to discover.

L232 · reusable dictionary

Line	Exact source (wrapped visually)	Meaning and purpose
232	<code>fn reusable_dictionary() {</code>	Named test wrapper generated by the suite macro; its body selects one concrete type and invokes a shared scenario.
233	<code> dictionary_clear::<\${d<i32, i32>>}();</code>	Invoke this shared scenario with the concrete type substituted by the suite macro; <...> chooses stored value types.
234	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
235	<code>#[test]</code>	Mark the following zero-argument function as a test for cargo test to discover.

L236 · owned dictionary

Line	Exact source (wrapped visually)	Meaning and purpose
236	<code>fn owned_dictionary() {</code>	Named test wrapper generated by the suite macro; its body selects one concrete type and invokes a shared scenario.
237	<code> dictionary_owned::<\$d<String, String>>());</code>	Invoke this shared scenario with the concrete type substituted by the suite macro; <...> chooses stored value types.
238	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
239	<code>};</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
240	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
241	<code>mod library {</code>	Start a test namespace so identical generated test names can coexist for each implementation.
242	<code> use super::*;</code>	Import the parent test module's helper functions, traits, and implementation types.
243	<code> suite!(LibraryStack, LibraryQueue, LibraryDictionary);</code>	Expand 16 named tests that use the standard-collection implementations.
244	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
245	<code>mod manual {</code>	Start a test namespace so identical generated test names can coexist for each implementation.
246	<code> use super::*;</code>	Import the parent test module's helper functions, traits, and implementation types.
247	<code> suite!(ManualStack, ManualQueue, ManualDictionary);</code>	Expand the identical 16 named tests using the manual implementations.
248	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
250	<code>#[test]</code>	Mark the following zero-argument function as a test for cargo test to discover.

L251 · deterministic differential operations

Scenario purpose: compare both versions after each reproducible mixed operation.

Line	Exact source (wrapped visually)	Meaning and purpose
251	<code>fn deterministic_differential_operations() {</code>	Generic test helper to compare both versions after each reproducible mixed operation. Its type bound permits either implementation.
252	<code> let mut a = LibraryDictionary::<i32, i32>::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
253	<code> let mut b = ManualDictionary::<i32, i32>::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
254	<code> let mut s1 = LibraryStack::<i32>::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
255	<code> let mut s2 = ManualStack::<i32>::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
256	<code> let mut q1 = LibraryQueue::<i32>::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
257	<code> let mut q2 = ManualQueue::<i32>::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
258	<code> let mut seed = 42_u32;</code>	Set a reproducible 32-bit unsigned seed; reruns use the same sequence.
259	<code> for step in 0..1000 {</code>	Perform 1,000 steps, numbered 0 through 999. The upper bound is excluded.
260	<code> seed = seed.wrapping_mul(1664525).wrapping_add(1013904223);</code>	Update a deterministic pseudo-random sequence; wrapping arithmetic avoids overflow panics.
261	<code> let key = ((seed >> 8) % 31) as i32;</code>	Shift bits, restrict the key to 0..30, then convert safely to a signed integer.
262	<code> match seed % 5 {</code>	Choose one of five operation codes from the deterministic sequence.
263	<code> 0 1 => {</code>	Either pattern 0 or 1 selects insertion/addition. The vertical bar means an alternative pattern.
264	<code> assert_eq!(a.insert(key, step), b.insert(key, step));</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: compare both versions after each reproducible mixed operation.
265	<code> s1.push(key);</code>	Apply the same mixed operation to this stack/queue implementation so its state can be compared with the other one.
266	<code> s2.push(key);</code>	Apply the same mixed operation to this stack/queue implementation so its state can be compared with the other one.

Line	Exact source (wrapped visually)	Meaning and purpose
267	<code>q1.enqueue(key);</code>	Apply the same mixed operation to this stack/queue implementation so its state can be compared with the other one.
268	<code>q2.enqueue(key);</code>	Apply the same mixed operation to this stack/queue implementation so its state can be compared with the other one.
269	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
270	<code>2 => {</code>	Operation code 2 selects deletion/removal from all three structures.
271	<code>assert_eq!(a.remove(&key), b.remove(&key));</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: compare both versions after each reproducible mixed operation.
272	<code>s2.pop(); assert_eq!(s1.pop(),</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: compare both versions after each reproducible mixed operation.
273	<code>q2.dequeue()); assert_eq!(q1.dequeue(),</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: compare both versions after each reproducible mixed operation.
274	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
275	<code>3 => assert_eq!(a.get(&key), b.get(&key)),</code>	Operation code 3 compares lookups without mutation.
276	<code>_ => {</code>	Catch all remaining operation codes; here that is 4, which clears all structures.
277	<code>a.clear();</code>	Clear this dictionary during the differential scenario; the matching instance receives the same operation.
278	<code>b.clear();</code>	Clear this dictionary during the differential scenario; the matching instance receives the same operation.
279	<code>s1.clear();</code>	Apply the same mixed operation to this stack/queue implementation so its state can be compared with the other one.
280	<code>s2.clear();</code>	Apply the same mixed operation to this stack/queue implementation so its state can be compared with the other one.
281	<code>q1.clear();</code>	Apply the same mixed operation to this stack/queue implementation so its state can be compared with the other one.
282	<code>q2.clear();</code>	Apply the same mixed operation to this stack/queue implementation so its state can be compared with the other one.

Line	Exact source (wrapped visually)	Meaning and purpose
283	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
284	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
285	assert_eq!(s1.peek(), s2.peek());	Require actual and expected values to be equal; failure panics and marks this test failed. Context: compare both versions after each reproducible mixed operation.
286	assert_eq!(q1.front(), q2.front());	Require actual and expected values to be equal; failure panics and marks this test failed. Context: compare both versions after each reproducible mixed operation.
287	assert_eq!((a.len(), s1.len(), q1.len()), (b.len(), s2.len(), q2.len()));	Require actual and expected values to be equal; failure panics and marks this test failed. Context: compare both versions after each reproducible mixed operation.
288	let mut x = Vec::new();	Allocate test-only output storage for the library dictionary traversal.
289	let mut y = Vec::new();	Allocate test-only output storage for the manual dictionary traversal.
290	a.visit(k, v x.push((*k, *v)));	Copy borrowed integer pairs into the library observation vector.
291	b.visit(k, v y.push((*k, *v)));	Copy borrowed integer pairs into the manual observation vector.
292	assert_eq!(x, y);	Require actual and expected values to be equal; failure panics and marks this test failed. Context: compare both versions after each reproducible mixed operation.
293	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
294	}	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
296	#[test]	Mark the following zero-argument function as a test for cargo test to discover.

L297 · manual values drop exactly once

Scenario purpose: check that every tracked owned value is destroyed exactly once.

Line	Exact source (wrapped visually)	Meaning and purpose
297	fn manual_values_drop_exactly_once() {	Generic test helper to check that every tracked owned value is destroyed exactly once. Its type bound permits either implementation.

Line	Exact source (wrapped visually)	Meaning and purpose
298	<code>use std::cell::Cell;</code>	Import interior-mutability storage for the test's destruction counter.
299	<code>use std::rc::Rc;</code>	Import a single-thread reference-counted pointer so tracked test values share the same counter.
300	<code>struct Tracked(Rc<Cell<usize>>);</code>	Define a tuple struct with one counter field; it deliberately does not implement Clone or Copy.
301	<code>impl Drop for Tracked {</code>	Implement automatic cleanup when this owner leaves scope or is explicitly dropped.

L302 · drop

Line	Exact source (wrapped visually)	Meaning and purpose
302	<code>fn drop(&mut self) {</code>	Automatic Drop hook, borrowing the soon-to-be-destroyed instance mutably.
303	<code>self.0.set(self.0.get() + 1);</code>	Read and increment the shared destruction counter; <code>.0</code> accesses the tuple struct's first field.
304	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
305	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
306	<code>let count = Rc::new(Cell::new(0));</code>	Create one shared counter initially set to zero.
307	<code>{</code>	Open a block; local owners created here are dropped when its scope ends.
308	<code>let mut s = ManualStack::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
309	<code>let mut q = ManualQueue::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
310	<code>let mut d = ManualDictionary::default();</code>	Construct an empty mutable test instance; type annotations or the generic helper determine its stored types.
311	<code>for key in 0..20 {</code>	Create 20 values in each manual structure; keys run from 0 through 19.
312	<code>s.push(Tracked(Rc::clone(&count)));</code>	Create a new tracked stack value sharing the counter; Rc cloning shares the pointer, not a stored value.
313	<code>q.enqueue(Tracked(Rc::clone(&count)));</code>	Create a new tracked queue value sharing the same counter.
314	<code>d.insert(key, Tracked(Rc::clone(&count)));</code>	Create a new tracked dictionary value for this key.
315	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
316	<code>drop(s.pop());</code>	Explicitly drop the removed stack value, immediately incrementing its counter.
317	<code>drop(q.dequeue());</code>	Explicitly drop the removed queue value.
318	<code>drop(d.remove(&10));</code>	Remove dictionary key 10 and immediately drop the returned value.

Line	Exact source (wrapped visually)	Meaning and purpose
319	<code>drop(d.insert(11, Tracked(Rc::clone(&count))));</code>	Create the 61st tracked value, replace key 11, and drop its old value immediately.
320	<code>assert_eq!(count.get(), 4);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: the selected shared scenario.
321	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.
322	<code>assert_eq!(count.get(), 61);</code>	Require actual and expected values to be equal; failure panics and marks this test failed. Context: the selected shared scenario.
323	<code>}</code>	Close the surrounding block or match arm. A comma separates arms; a semicolon terminates a declaration when present.

13. Configuration, verification, and your next steps

Cargo.toml, line by line

Line	Configuration	Explanation
1	[package]	Begin package metadata.
2	name = "rust_structures_assignment"	Cargo package name; the binary/tests also use it as the library import name.
3	version = "0.1.0"	Initial semantic version for this learning project.
4	edition = "2021"	Select Rust 2021 syntax and compatibility rules; this is not the installed compiler version.
5	rust-version = "1.70"	Declare the supported minimum compiler version; that version was used for verification.
6	description = "Library-backed and manually implemented stack, queue, and ordered dictionary"	Human-readable package purpose.
8	[dependencies]	Begin dependency declarations. The section is empty: there are no external crates.

What the other files do

Cargo.lock records dependency resolution and is generated by Cargo; this project has only its own package entry. README.md explains execution and design. TEST_CASES.md describes expected behavior. AI_USAGE.md preserves exact user prompts and discloses generated work. DEMO_OUTPUT.txt and TEST_OUTPUT.txt record actual execution output. SOURCE_MANIFEST.json records source hashes so you can detect drift from this guide. LINE_BY_LINE.json supplies the same annotations in a searchable machine-readable form.

Verified checks

Check	Observed result
cargo test	34 passed, 0 failed. Library/binary unit-test targets and doc-tests contain 0 tests; the 34 scenarios are integration tests.
cargo run --quiet	Both versions printed matching data-structure results.
cargo fmt --check	Passed against the final formatted source.
cargo clippy --all-targets -- -D warnings	Passed with warnings treated as errors.
Compiler	rustc 1.70.0 (90c541806 2023-05-31).
Cargo	cargo 1.70.0 (ec8a8a0ca 2023-04-25).

Verification used the declared minimum Rust version, 1.70.0. The source uses stable language features and requires no external crates. The tests establish the listed behaviors in the exercised scenarios; they do not prove correctness for every possible input or measure performance.

Practice without changing the assignment behavior

Predict before running

Change the demo values to 4,4,9. Predict both removal orders. Duplicates are separate stack/queue entries.

Trace a missing key

Search for 0 in the illustrated tree. Explain the comparisons and why an empty link yields None.

Explain a duplicate key

Insert total twice. Explain why dictionary size remains unchanged and why insert returns the old value.

Explain a successor

Draw the 8,4,12,10,11,14 tree. Delete 8 and show where node 11 goes.

Learn borrowing

Explain why peek returns a reference while pop returns an owned value. Do not add cloning just to bypass this distinction.

Understand complexity

Describe why the manual queue traverses all nodes on insertion and why sorted keys can make the manual tree slow.

Read the macros last

Follow one suite expansion: choose LibraryStack, substitute it for \$s, and find the resulting empty_stack test.

14. Official references and submission checklist

These links are primary Rust documentation. The tutorial explanations and code are original to this delivery; the links let you check language and collection semantics directly.

[R1] Ownership

<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html>

[R2] References and borrowing

<https://doc.rust-lang.org/book/ch04-02-references-and-borrowing.html>

[R3] Box and recursive types

<https://doc.rust-lang.org/book/ch15-01-box.html>

[R4] Option methods

<https://doc.rust-lang.org/std/option/enum.Option.html>

[R5] Traits

<https://doc.rust-lang.org/book/ch10-02-traits.html>

[R6] Standard collections

<https://doc.rust-lang.org/std/collections/index.html>

[R7] Vec

<https://doc.rust-lang.org/std/vec/struct.Vec.html>

[R8] VecDeque

<https://doc.rust-lang.org/std/collections/struct.VecDeque.html>

[R9] BTreeMap

<https://doc.rust-lang.org/std/collections/struct.BTreeMap.html>

[R10] Writing tests

<https://doc.rust-lang.org/book/ch11-01-writing-tests.html>

Before uploading to Canvas

1. Run the project on your own computer. 2. Read the AI disclosure and retain the exact prompts. 3. Check whether the instructor intended sorted-key order or a hash table specifically. 4. Keep the source and documentation in your mini-project Git repository. 5. Submit the requested code and test descriptions; attach this guide if useful. The supplied Git bundle provides local history, but no remote push or Canvas upload has been performed.